

Numerical Analysis Final Project

B11501015 廖致翔

Q1:

(a) 令 α 為 L_1 的參數、 β 為 L_2 的參數，則

$$P(\alpha) = \begin{bmatrix} 5 \\ -7 \\ 1 \end{bmatrix} + \alpha \begin{bmatrix} 3 \\ -6 \\ -2 \end{bmatrix} = \begin{bmatrix} 5+3\alpha \\ -7-6\alpha \\ 1-2\alpha \end{bmatrix}, Q(\beta) = \begin{bmatrix} 1 \\ 0 \\ -5 \end{bmatrix} + \beta \begin{bmatrix} 3 \\ 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 1+3\beta \\ 2\beta \\ -5+2\beta \end{bmatrix}, \text{ 因此}$$

$$P(\alpha) - Q(\beta) = \begin{bmatrix} 4+3\alpha-3\beta \\ -7-6\alpha-2\beta \\ 6-2\alpha-2\beta \end{bmatrix} \text{ 距離為 } \overline{PQ}(\alpha, \beta) = \|P(\alpha) - Q(\beta)\|_2, \text{ 取距離平方作}$$

為 cost function : $J(\alpha, \beta) = \|P(\alpha) - Q(\beta)\|_2^2$ ，因為平方根函數在正數上單調遞

增，所以最小化 J 和最小化 $\overline{PQ}$ 會得到相同的 α 與 β 。將上式展開可得

$$J(\alpha, \beta) = (4+3\alpha-3\beta)^2 + (-7-6\alpha-2\beta)^2 + (6-2\alpha-2\beta)^2$$

$$= 49\alpha^2 + 17\beta^2 + 14\alpha\beta + 84\alpha - 20\beta + 101$$

$$\overline{PQ}(\alpha, \beta) = \|P(\alpha) - Q(\beta)\|_2 = \sqrt{49\alpha^2 + 17\beta^2 + 14\alpha\beta + 84\alpha - 20\beta + 101}$$

(b)

令 J 對 α 與 β 的偏微分為零： $\frac{\partial J}{\partial \alpha} = 98\alpha + 14\beta + 84 = 0, \frac{\partial J}{\partial \beta} = 14\alpha + 34\beta - 20 = 0$ 。化

簡後得到 $\begin{cases} 7\alpha + \beta = -6, \\ 7\alpha + 17\beta = 10. \end{cases}$ 解得 $\alpha = -1, \beta = 1$ ，代回兩條直線的參數式：

$$P = P(-1) = \begin{bmatrix} 5 \\ -7 \\ 1 \end{bmatrix} - \begin{bmatrix} 3 \\ -6 \\ -2 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix}, Q = Q(1) = \begin{bmatrix} 1 \\ 0 \\ -5 \end{bmatrix} + \begin{bmatrix} 3 \\ 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \\ -3 \end{bmatrix}, P - Q = \begin{bmatrix} -2 \\ -3 \\ 6 \end{bmatrix} \text{ 所以最}$$

小 cost 與最小距離分別為 $J_{\min} = (-2)^2 + (-3)^2 + 6^2 = 49, \overline{PQ}_{\min} = \sqrt{49} = 7$

(c)先令兩點重合， $P(\alpha)=Q(\beta)$ ，由參數式可得
$$\begin{bmatrix} 5 \\ -7 \\ 1 \end{bmatrix} + \alpha \begin{bmatrix} 3 \\ -6 \\ -2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ -5 \end{bmatrix} + \beta \begin{bmatrix} 3 \\ 2 \\ 2 \end{bmatrix}$$

$$\Rightarrow \begin{bmatrix} 3 & -3 \\ -6 & -2 \\ -2 & -2 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} -4 \\ 7 \\ -6 \end{bmatrix}, \text{ 記為 } Ax=b, \quad A = \begin{bmatrix} 3 & -3 \\ -6 & -2 \\ -2 & -2 \end{bmatrix}, \quad x = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}, \quad b = \begin{bmatrix} -4 \\ 7 \\ -6 \end{bmatrix}.$$

這是一個三條方程、兩個未知數的 overdetermined system。這樣的系統不一定有精確解。此 $Ax=b$ 沒有精確解，也就是兩條直線不相交，P 與 Q 重合無解。

(d)依照 least-squares solution，改尋找使殘差最小的參數： $\min_x \|Ax-b\|_2^2$ 。

令 $J(x) = (Ax-b)^T(Ax-b)$ ，對 x 微分並令 J 為零，可以得到 normal equations：

$A^T Ax = A^T b$ ，若 $A^T A$ 可逆，則 least-squares 解為 $x = (A^T A)^{-1} A^T b \equiv A_L^+ b$ ，其中

$A_L^+ = (A^T A)^{-1} A^T$ 為 left pseudo-inverse。

$$A^T A = \begin{bmatrix} 49 & 7 \\ 7 & 17 \end{bmatrix}, \quad A^T b = \begin{bmatrix} -42 \\ 10 \end{bmatrix}, \text{ 因此 } \begin{bmatrix} 49 & 7 \\ 7 & 17 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} -42 \\ 10 \end{bmatrix}。由於$$

$$\det(A^T A) = 49 \cdot 17 - 7 \cdot 7 = 784 \neq 0, \text{ 所以解唯一， } \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}。$$

這與直接最小化距離平方得到的參數完全相同。此時的最小殘差為

$$Ax-b = \begin{bmatrix} 3 & -3 \\ -6 & -2 \\ -2 & -2 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} - \begin{bmatrix} -4 \\ 7 \\ -6 \end{bmatrix} = \begin{bmatrix} -2 \\ -3 \\ 6 \end{bmatrix}, \text{ 其長度為 } \|Ax-b\|_2 = 7, \text{ 正是兩條直線之間}$$

的最短距離。

$$(e) \text{ 令 } p_0 = \begin{bmatrix} 5 \\ -7 \\ 1 \end{bmatrix}, \quad q_0 = \begin{bmatrix} 1 \\ 0 \\ -5 \end{bmatrix}, \quad u = \begin{bmatrix} 3 \\ -6 \\ -2 \end{bmatrix}, \quad v = \begin{bmatrix} 3 \\ 2 \\ 2 \end{bmatrix}, \text{ 則}$$

$$P(\alpha)-Q(\beta) = p_0 - q_0 + \alpha u - \beta v, \text{ 若定義 } A = [u \quad -v], x = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}, b = q_0 - p_0, \text{ 則}$$

$$Ax-b = u\alpha - v\beta - (q_0 - p_0) = p_0 - q_0 + \alpha u - \beta v = P(\alpha) - Q(\beta). \text{ 因此}$$

$\|P(\alpha) - Q(\beta)\|_2^2 = \|Ax - b\|_2^2$ ，也就是說，直接最小化距離平方，其實就是在最小化線性系統 $Ax = b$ 的殘差平方。對此微分，兩者都會得到 $A^T Ax = A^T b$ ，所以只要 $A^T A$ 可逆，兩種方法會求出同一組唯一參數。此題的 $\det(A^T A) = 784 \neq 0$ ，因此解唯一，且兩種方法完全等價。

Q2:

(a) 解析解

先求齊次方程 $y'' + 4y' + 4y = 0$ ，其特徵方程為 $r^2 + 4r + 4 = (r + 2)^2 = 0$ ，所以

$r = -2$ 是重根，齊次解為 $y_h(x) = (C_1 + C_2 x)e^{-2x}$ 。由於非齊性項為 x ，令 particular

solution 為 $y_p(x) = ax + b$ 代入原方程得 $0 + 4a + 4(ax + b) = x$ ，比較係數，可得

$4a = 1$ ， $4a + 4b = 0$ 。因此 $a = \frac{1}{4}$ ， $b = -\frac{1}{4}$ ， $y_p(x) = \frac{x-1}{4}$ 。故通解為

$y(x) = (C_1 + C_2 x)e^{-2x} + \frac{x-1}{4}$ 。接著套入邊界條件。由 $y(0) = 0$ ，得到 $C_1 - \frac{1}{4} = 0$

， $C_1 = \frac{1}{4}$ 。由 $y(1) = 2$ ，得到 $(C_1 + C_2)e^{-2} = 2$ ， $C_1 + C_2 = 2e^2$ 。因此 $C_2 = 2e^2 - \frac{1}{4}$ 。

最後解析解為 $y_{\text{exact}}(x) = \left[\frac{1}{4} + \left(2e^2 - \frac{1}{4} \right) x \right] e^{-2x} + \frac{x-1}{4}$

(b) Finite Difference Method

此題為 Dirichlet boundary conditions，因此格點直接在邊界上：

$x_0 = 0$ ， $x_{N-1} = 1$ ，未知量只有點 $y_1, y_2, \dots, y_{N-2}$ 。在 x_i ， $(i = 1, 2, \dots, N-2)$ ，使

用二階中心差分 $y''(x_i) \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{\Delta x^2}$ ，以及 $y'(x_i) \approx \frac{y_{i+1} - y_{i-1}}{2\Delta x}$ ，代回原方程得到

$\frac{y_{i+1} - 2y_i + y_{i-1}}{\Delta x^2} + 4\frac{y_{i+1} - y_{i-1}}{2\Delta x} + 4y_i = x_i$ ，整理後可寫成三對角線性系統：

$\left(\frac{1}{\Delta x^2} - \frac{2}{\Delta x} \right) y_{i-1} + \left(-\frac{2}{\Delta x^2} + 4 \right) y_i + \left(\frac{1}{\Delta x^2} + \frac{2}{\Delta x} \right) y_{i+1} = x_i$ 。令

$a = \frac{1}{\Delta x^2} - \frac{2}{\Delta x}$ ， $b = -\frac{2}{\Delta x^2} + 4$ ， $c = \frac{1}{\Delta x^2} + \frac{2}{\Delta x}$ ，則方程為 $ay_{i-1} + by_i + cy_{i+1} = x_i$ 。邊界值

$y_0 = 0$ 、 $y_{N-1} = 2$ 移到右邊，因此第一列與最後一列分別為 $by_1 + cy_2 = x_1 - ay_0$ ，

$ay_{N-3} + by_{N-2} = x_{N-2} - cy_{N-1}$ ，得到 $Ay = d$ ，形式如下：

$$\begin{bmatrix} b & c & 0 & \cdots & 0 & 0 \\ a & b & c & \cdots & 0 & 0 \\ 0 & a & b & \ddots & 0 & 0 \\ \vdots & \vdots & \ddots & \ddots & c & 0 \\ 0 & 0 & 0 & a & b & c \\ 0 & 0 & 0 & 0 & a & b \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{N-3} \\ y_{N-2} \end{bmatrix} = \begin{bmatrix} x_1 - ay_0 \\ x_2 \\ x_3 \\ \vdots \\ x_{N-3} \\ x_{N-2} - cy_{N-1} \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{N-3} \\ x_{N-2} - 2c \end{bmatrix} \quad \circ \text{程式中使用 Gaussian}$$

elimination with partial pivoting。由於使用二階中心差分，預期有限差分法的主要 truncation error 為 $\|e\|_{\infty} = O(\Delta x^2)$ 。 (iii). 數值解與精確解比較在(d)部分)

(c) Shooting method

令 $Y(x) = \begin{bmatrix} y(x) \\ z(x) \end{bmatrix}$, $z(x) = y'(x)$ ，則原方程可化為一階系統

$$\frac{dY}{dx} = F(x, Y) = MY + g(x) = \begin{bmatrix} 0 & 1 \\ -4 & -4 \end{bmatrix} \begin{bmatrix} y(x) \\ z(x) \end{bmatrix} + \begin{bmatrix} 0 \\ x \end{bmatrix}, \text{ 由於這是一個線性 BVP, 可}$$

拆成兩個 IVP。令 u 為非齊次 IVP 的解： $\frac{du}{dx} = Mu + g(x), u(0) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ ，令 v 為齊次

IVP 的解： $\frac{dv}{dx} = Mv, v(0) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ 。則 BVP 的解可寫成 $Y(x) = u(x) + \theta v(x)$ ，其中 θ

代表未知初始斜率的調整量。利用右端邊界條件 $y(1) = 2$ ，可得 $u_1(1) + \theta v_1(1) = 2$

因此 $\theta = \frac{2 - u_1(1)}{v_1(1)}$ ，最後數值解為 $y_{\text{shoot}}(x) = u_1(x) + \theta v_1(x)$ 。

(i) 第一種 time-marching scheme : Forward Euler method

$Y_{i+1} = Y_i + \Delta x F(x_i, Y_i)$ ，Forward Euler 的 local truncation error 為 $O(\Delta x^2)$ ，在固定區間上累積後 global error 為 $O(\Delta x)$ ，因此 shooting method 若用 Forward Euler 解兩個 IVP，預期整體誤差約為一階。

(ii) 第二種 time-marching scheme : Crank--Nicolson method

對 IVP 右邊做 trapezoidal rule： $Y_{i+1} = Y_i + \frac{\Delta x}{2} [F(x_i, Y_i) + F(x_{i+1}, Y_{i+1})]$ ，因為此題

$F(x, Y) = MY + g(x)$ 是線性的，上式可以整理為：

$$\left(I - \frac{\Delta x}{2} M \right) Y_{i+1} = \left(I + \frac{\Delta x}{2} M \right) Y_i + \frac{\Delta x}{2} [g(x_i) + g(x_{i+1})], \text{ 程式中把每一步的 } 2 \times 2 \text{ 線}$$

性系統直接寫成公式。若令 $Y_i = (y_i, z_i)^T$ ，則每一步在解：

$$\begin{bmatrix} 1 & -\Delta x / 2 \\ 2\Delta x & 1 + 2\Delta x \end{bmatrix} \begin{bmatrix} y_{i+1} \\ z_{i+1} \end{bmatrix} = \begin{bmatrix} y_i + \frac{\Delta x}{2} z_i \\ z_i - 2\Delta x y_i - 2\Delta x z_i + \frac{\Delta x}{2} (s_i + s_{i+1}) \end{bmatrix}, \text{ 其中非齊次問題取}$$

$s_i = x_i$ ，齊次問題取 $s_i = 0$ 。Crank--Nicolson 的 global error 為 $O(\Delta x^2)$ ，因此預期

使用 Crank--Nicolson 的 shooting method 會呈現二階收斂。

(iii.數值解與精確解比較在(d)部分)

(d)誤差分析

令 $e_i = y_i - y_{\text{exact}}(x_i)$ ，主要使用 maximum absolute error： $\|e\|_{\infty} = \max_i |e_i|$ ，另外

也列出 discrete L^2 error 與 RMS error： $\|e\|_{2,h} = \left(\Delta x \sum_{i=0}^{N-1} e_i^2 \right)^{1/2}$ ， $e_{\text{RMS}} = \left(\frac{1}{N} \sum_{i=0}^{N-1} e_i^2 \right)^{1/2}$

觀察收斂階數時，使用相鄰兩組網格的誤差估計 $p \approx \frac{\log(E_{\Delta x_1} / E_{\Delta x_2})}{\log(\Delta x_1 / \Delta x_2)}$

數值結果與討論：

程式使用 $N = 11, 21, 41, 81, 161, 321$ ，對應 $\Delta x = \frac{1}{10}, \frac{1}{20}, \frac{1}{40}, \frac{1}{80}, \frac{1}{160}, \frac{1}{320}$ 。

method	N_grid	Delta_x	max_abs_error	discrete_l2_error	rms_error	endpoint_error	theta
{'Finite difference'}	1.10000000000000e+01	1.00000000000000e-01	3.02704295197875e-02	1.97806877988757e-02	1.88601458057748e-02	0.00000000000000e+00	NaN
{'Shooting Forward Euler'}	1.10000000000000e+01	1.00000000000000e-01	4.21460932795490e-01	2.79959977065359e-01	2.66931364617873e-01	0.00000000000000e+00	1.44511611938477e+01
{'Shooting Crank-Nicolson'}	1.10000000000000e+01	1.00000000000000e-01	1.12569842861765e-02	7.53095065135622e-03	7.18047976752288e-03	0.00000000000000e+00	1.42312858392538e+01
{'Finite difference'}	2.10000000000000e+01	5.00000000000000e-02	7.47583782447814e-03	4.86348633460704e-03	4.74619859672137e-03	0.00000000000000e+00	NaN
{'Shooting Forward Euler'}	2.10000000000000e+01	5.00000000000000e-02	1.86585463378925e-01	1.24471580651616e-01	1.21471832416852e-01	0.00000000000000e+00	1.43304740011495e+01
{'Shooting Crank-Nicolson'}	2.10000000000000e+01	5.00000000000000e-02	2.79648591647241e-03	1.87188055148504e-03	1.82676836674519e-03	0.00000000000000e+00	1.42664179946010e+01
{'Finite difference'}	4.10000000000000e+01	2.50000000000000e-02	1.86428934326077e-03	1.21873889206730e-03	1.1958263759960e-03	0.00000000000000e+00	NaN
{'Shooting Forward Euler'}	4.10000000000000e+01	2.50000000000000e-02	8.82444501777893e-02	5.89737468556630e-02	5.82501151946590e-02	4.44889209850063e-16	1.42970935418372e+01
{'Shooting Crank-Nicolson'}	4.10000000000000e+01	2.50000000000000e-02	6.98020807728295e-04	4.67263808082701e-04	4.61530551950254e-04	0.00000000000000e+00	1.42751894178485e+01
{'Finite difference'}	8.10000000000000e+01	1.25000000000000e-02	4.65665201405227e-04	3.02365112206464e-04	3.00492364408004e-04	0.00000000000000e+00	NaN
{'Shooting Forward Euler'}	8.10000000000000e+01	1.25000000000000e-02	4.29735681688546e-02	2.87350755669813e-02	2.85571476917171e-02	0.00000000000000e+00	1.42859418391354e+01
{'Shooting Crank-Nicolson'}	8.10000000000000e+01	1.25000000000000e-02	1.74461084470945e-04	1.16771384996481e-04	1.16048335412858e-04	0.00000000000000e+00	1.42773815509813e+01
{'Finite difference'}	1.61000000000000e+02	6.25000000000000e-03	1.16405575746992e-04	7.55713012037896e-05	7.53362421547319e-05	0.00000000000000e+00	NaN
{'Shooting Forward Euler'}	1.61000000000000e+02	6.25000000000000e-03	2.12091408870224e-02	1.41868570308039e-02	1.41427298411860e-02	0.00000000000000e+00	1.42816269149823e+01
{'Shooting Crank-Nicolson'}	1.61000000000000e+02	6.25000000000000e-03	4.36194875197415e-05	2.91908529570782e-05	2.90992594153402e-05	0.00000000000000e+00	1.42779295391481e+01
{'Finite difference'}	3.21000000000000e+02	3.12500000000000e-03	2.98993546903096e-05	1.88915767996278e-05	1.88621277142261e-05	0.00000000000000e+00	NaN
{'Shooting Forward Euler'}	3.21000000000000e+02	3.12500000000000e-03	1.0536059657398e-02	7.04912670626473e-03	7.03813819341679e-03	0.00000000000000e+00	1.42797715145707e+01
{'Shooting Crank-Nicolson'}	3.21000000000000e+02	3.12500000000000e-03	1.09046031440130e-05	7.29733854814889e-06	7.28596311091624e-06	0.00000000000000e+00	1.42780665333709e+01

▲ Fig.1 Matlab code 輸出之不同方法在各網格下的 error(整理成下表)

Table 1: 不同方法在各網格下的 maximum absolute error

N	Δx	Method	$\ e\ _{\infty}$
11	0.100000	Finite difference	3.027×10^{-2}
11	0.100000	Shooting: Forward Euler	4.215×10^{-1}
11	0.100000	Shooting: Crank–Nicolson	1.126×10^{-2}
21	0.050000	Finite difference	7.476×10^{-3}
21	0.050000	Shooting: Forward Euler	1.866×10^{-1}
21	0.050000	Shooting: Crank–Nicolson	2.797×10^{-3}

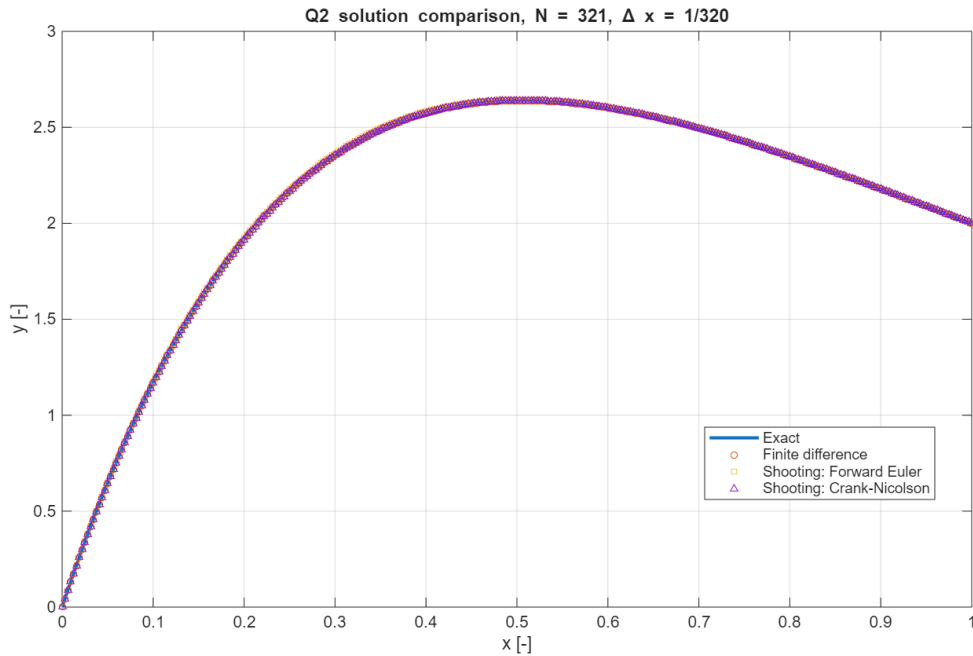
41	0.025000	Finite difference	1.864×10^{-3}
41	0.025000	Shooting: Forward Euler	8.824×10^{-2}
41	0.025000	Shooting: Crank–Nicolson	6.980×10^{-4}
81	0.012500	Finite difference	4.657×10^{-4}
81	0.012500	Shooting: Forward Euler	4.297×10^{-2}
81	0.012500	Shooting: Crank–Nicolson	1.745×10^{-4}
161	0.006250	Finite difference	1.164×10^{-4}
161	0.006250	Shooting: Forward Euler	2.121×10^{-2}
161	0.006250	Shooting: Crank–Nicolson	4.362×10^{-5}
321	0.003125	Finite difference	2.910×10^{-5}
321	0.003125	Shooting: Forward Euler	1.054×10^{-2}
321	0.003125	Shooting: Crank–Nicolson	1.091×10^{-5}

Command Window				
N_grid	Delta_x	order_FD	order_shoot_FE	order_shoot_CN
2.10000000000000e+01	5.00000000000000e-02	2.01760196858162e+00	1.17556231506077e+00	2.00913341548985e+00
4.10000000000000e+01	2.50000000000000e-02	2.00360948468119e+00	1.08025913893311e+00	2.00227311634496e+00
8.10000000000000e+01	1.25000000000000e-02	2.00126080747502e+00	1.03805597651931e+00	2.00036478046852e+00
1.61000000000000e+02	6.25000000000000e-03	2.00013291134986e+00	1.01874978392995e+00	1.99986053764891e+00
3.21000000000000e+02	3.12500000000000e-03	2.00010109872448e+00	1.00936473658399e+00	2.00003555371985e+00

▲ Fig.2 Matlab code 輸出之由相鄰網格估計的 observed order (整理成下表)

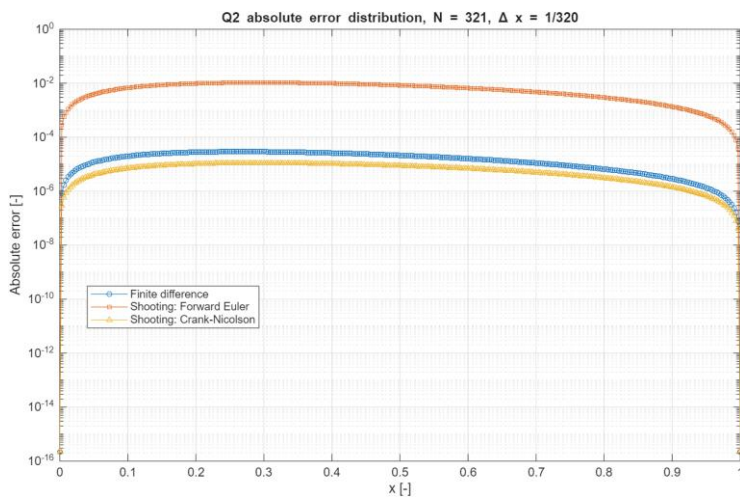
Table 2

N	Δx	Finite difference	Forward Euler shooting	Crank–Nicolson shooting
21	0.050000	2.0176	1.1756	2.0091
41	0.025000	2.0036	1.0803	2.0023
81	0.012500	2.0013	1.0381	2.0004
161	0.006250	2.0001	1.0187	1.9999
321	0.003125	2.0001	1.0094	2.0000



▲ Fig.3 在 $N=321$ 、 $\Delta x = 1/320$ 時，解析解與三種數值解的比較

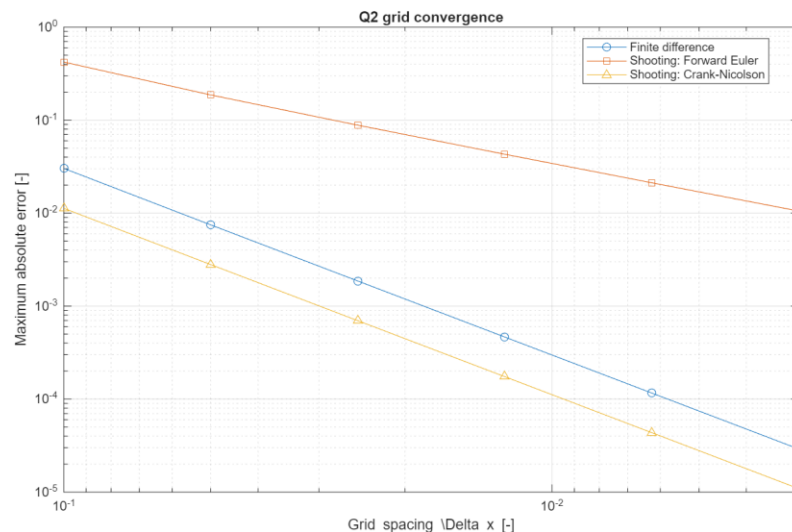
從 Figure 3 可以看到，當網格細到 $\Delta x = 1/320$ 時，finite difference、Forward Euler shooting、Crank--Nicolson shooting 在整體曲線上都能捕捉正確趨勢。解由 $x=0$ 時的 $y=0$ 上升，在約 $x \approx 0.5$ 附近達到最大值，之後逐漸下降至 $y(1)=2$ 。Forward Euler 的誤差雖然最大，但在此細網格下，已不容易用肉眼分辨。



▲ Fig.4 在 $N=321$ 、 $\Delta x = 1/320$ 時，各方法的 pointwise absolute error。端點因為邊界條件直接滿足，所以誤差接近 0

Figure 4 顯示更清楚的誤差分布。Forward Euler shooting 的誤差大約落在 10^{-2} 量級，主要原因是 Forward Euler 只有一階 global accuracy；finite difference 的誤差約落在 10^{-5} 到 10^{-4} 之間；Crank--Nicolson shooting 則約落在 10^{-5} 量級，且比 finite difference 更小一些。兩個二階方法的 error constant 不同。此題中 Crank--

Nicolson shooting 的誤差常數較小，因此同樣二階收斂下會出現較低的誤差曲線。



▲ Fig.5 Maximum absolute error 對 grid spacing Δx 的 log scale 收斂圖

Figure 5 當 Δx 每次減半時，finite difference 的 maximum error 約減少四倍，observed order 幾乎等於 2；Forward Euler shooting 的 error 約減半，observed order 逐漸趨近於 1；Crank--Nicolson shooting 的 error 也約減少四倍，observed order 幾乎等於 2。這與前述理論一致：

Finite difference = $O(\Delta x^2)$, Forward Euler shooting = $O(\Delta x)$, Crank--Nicolson shooting = $O(\Delta x^2)$

此題的主要誤差來自離散化。Finite difference method 的誤差來自二階中心差分對 y' 與 y'' 的 truncation error，因此呈現 $O(\Delta x^2)$ 。Shooting method 的誤差則主要來自 IVP time-marching scheme；Forward Euler 的累積誤差為 $O(\Delta x)$ ，Crank--Nicolson 的累積誤差為 $O(\Delta x^2)$ 。此外，線性系統解法與浮點運算會有 round-off error，但此題矩陣規模不大，且使用 double precision 與 partial pivoting，因此 round-off error 遠小於 discretization error。圖中的端點誤差接近 10^{-16} ，是因為 Dirichlet 邊界條件與 shooting parameter θ 直接使兩端邊界被滿足

程式說明:首先定義解析解 $y_{\text{exact}}(x)$ ，在 finite difference 部分，程式建立三對角矩陣，將兩端 Dirichlet 邊界值移入右手邊，最後使用自寫的 Gaussian elimination with partial pivoting 解出內點未知量。沒有使用 MATLAB 的 backslash function。

在 shooting method 部分，程式分別計算非齊次 IVP u 與齊次 IVP v ，再由

$\theta = \frac{2 - u_1(1)}{v_1(1)}$ 組合出滿足 $y(1)=2$ 的解。Forward Euler 直接使用

$Y_{i+1} = Y_i + \Delta x F(x_i, Y_i)$ ，而 Crank--Nicolson 則在每一步手動解 2×2 線性系統。最

後程式輸出誤差表、observed order 表、solution comparison、pointwise error distribution 與 grid convergence 圖，用來檢查數值解是否符合預期收斂階數。

Q3:

(0) 基礎假設

(i) 幾何、材料參數與符號統一

$P_0 = 8\text{kN} = 8000\text{N}$, $L = 60\text{mm}$, $a = 10\text{mm}$, $b = 1\text{mm}$, $E = 100\text{GPa} = 100000\text{N/mm}^2$, $\nu = 0.3$

採用 mm--N 單位系，所有幾何長度用 mm，力用 N，應力與彈性模數用

N/mm^2 。且 $I = \frac{2(1)(10)^3}{3} = \frac{2000}{3}\text{mm}^4$

題目給定 Timoshenko--Goodier 懸臂梁位移解析式為：

$$u_1(x_1, x_2) = -\frac{P_s}{2EI}(x_1^2 - L^2)x_2 + \frac{\nu P_s}{6EI}x_2^3 - \frac{P_s}{2GI}\left(a^2x_2 - \frac{x_2^3}{3}\right),$$

$$u_2(x_1, x_2) = \frac{P_s}{6EI}(x_1^3 - 3L^2x_1 + 2L^3) + \frac{\nu P_s}{2EI}(L - x_1)x_2^2 + \frac{P_s a^2}{2GI}(L - x_1)$$

在自由端中心點 $(x_1, x_2) = (0, 0)$ ，垂直位移為 $u_2(0, 0) = \frac{P_s L^3}{3EI} + \frac{P_s a^2 L}{2GI} = -9.576\text{mm}$

根據新版題目：總外力為 $P_s = -8000\text{N}$ 。自由端面上的均勻力可寫成 $T_2 = \frac{P_s}{2ab}$ 。

對每個位於自由端的邊界元素，外力向量由 $\mathbf{F}_e^e = b \int_{\Gamma_e} \mathbf{N}^T \mathbf{T} dL$ 計算。由於 T_2 為常數，且邊界上的一維線性形狀函數會將該邊界元素的總力平均分配給上下兩個端點，因此每個自由端邊界元素的兩個端點各得到 $\frac{P_s}{2N_y}$ 的垂直力。這正好對應

新版題目給出的元素力向量 $\mathbf{F}^e$ 。

(ii) 誤差定義

舊版題目：程式用 point_center，也就是在左端中心節點施加集中力。集中力在連續問題中視為 delta-type loading，會造成局部解的規律性降低；同時有限元素程式以右端整條邊 zero_clamp 強制 $u_1 = u_2 = 0$ 。因此，FEM 主模型與題目提供的解析場在局部邊界條件上不完全是同一個連續問題。

因此使用兩類誤差： $e_{\text{exact}}(h) = \frac{\|u_h - u_{\text{exact}}\|_{L^2}}{\|u_{\text{exact}}\|_{L^2}}$ ，以及 $e_{\text{ref}}(h) = \frac{\text{RMS}(u_h - u_{\text{finest}})}{\text{RMS}(u_{\text{finest}})}$ ，第一

個誤差衡量 FEM 解與解析場的差距；第二個誤差則以最細網格解為參考，觀察

同一個 point-load FEM 模型本身的 mesh convergence。如此可以區分「模型差異」以及「離散誤差」。

(1)

由弱式推導可知，每一個元素剛性矩陣的形式是 $K^e = \int_{D_e} B^T D B dV$ 。B 表示

「節點位移會在元素內造成多少應變」，D 表示「材料如何把應變轉成應力」，而 B^T 則把元素內部的材料反力投影回節點自由度。因此， K^e 是元素內彈性應變能對節點位移的二次型表示。

(i)對稱性： K^e 對稱，組裝後 K 也對稱

因為平面應力材料矩陣 D 是對稱矩陣，i.e. $D = D^T$ ，所以

$$(K^e)^T = \left(\int_{D_e} B^T D B dV \right)^T = \int_{D_e} B^T D^T B dV = K^e$$
，而元素組裝只是依照 local DOF

與 global DOF 的對應關係，把每個 K^e 加到全域矩陣的對應位置。這個加法過程不會破壞對稱性，因此 assembled stiffness matrix K 必須滿足 $K = K^T$

在程式中，用 $\frac{\|K - K^T\|_F}{\max(\|K\|_F, \epsilon)}$ 檢查組裝後的相對對稱誤差，並且在 assembly 後使

用 $K = 0.5 * (K + K^T)$ 消除浮點數累積造成的非對稱誤差。實際程式輸出的 K_symmetry_relative_error_col 為 0，驗證了 assembly 的對稱性。

(ii)未施加固定端 boundary condition 前，K 是 positive semidefinite

線彈性問題的内能可寫成 $\frac{1}{2} U^T K U = \frac{1}{2} \sum_e \int_{D_e} \epsilon_h^T D \epsilon_h dV$ ，由於材料矩陣 D 正定，

只要位移場造成非零應變，右式就會大於零；因此 $U^T K U \geq 0$ (半正定)。可是在還沒有施加固定端 BC 以前，整根梁可以做剛體平移或剛體轉動。這些 rigid body 的位移向量不是零，但它們在元素內不造成應變，所以彈性能量為零。也就是說，存在某些非零 U 使得 $U^T K U = 0$ ，因此，未施加固定端 boundary condition 前，K 是 positive semidefinite，並且具有 null space。這也解釋了為什麼有限元素法不能只把所有元素矩陣組裝起來就直接解 $KU = F$ ；若沒有適當固定邊界，線性系統會因剛體模態而奇異或接近奇異。

(iii)施加固定端 BC 後：自由自由度矩陣 K_{ff} 是對稱正定

固定端位於 $x_1 = L$ ，程式中的 zero_clamp 將固定端所有節點的 u_1, u_2 都設為零。

這移除了二維平面問題中的兩個剛體平移自由度與一個剛體轉動自由度。若將自由 DOF(以 f 表示)與固定 DOF(以 c 表示)分開，線性系統可以寫成

$$\begin{bmatrix} K_{ff} & K_{fc} \\ K_{cf} & K_{cc} \end{bmatrix} \begin{bmatrix} U_f \\ U_c \end{bmatrix} = \begin{bmatrix} F_f \\ F_c \end{bmatrix}$$
，因固定端位移 U_c 已知，需解的是 $K_{ff} U_f = F_f - K_{fc} U_c$

本題中 $U_c = 0$ ，所以 $K_{ff}U_f = F_f$ 。在材料、元素沒有退化、固定條件足以消除

剛體模態的情況下，自由 DOF 矩陣會滿足 $K_{ff} = K_{ff}^T$ ， $z^T K_{ff} z > 0$ for all $z \neq 0$ ，

因此 K_{ff} 是 sparse symmetric positive definite。

程式輸出驗證：每組網格的 chol_flag_free_col 都是 0，代表 K_{ff} 的 Cholesky

factorization 成功；這代表 K_{ff} 具有 SPD 結構。

(iv) 稀疏性：局部支撐造成 sparse matrix

每個 Q4 元素只有四個節點，每個節點有兩個自由度，因此元素剛性矩陣 K^e 是 8×8 。一個元素只會讓同一元素內的八個自由度彼此耦合；相距很遠的節點不會直接出現在同一個元素矩陣裡。因此組裝後，每個節點只和相鄰元素中的節點互相耦合，全域矩陣絕大多數位置都是零。這就是 FEM 剛性矩陣的 sparsity。

程式輸出驗證：

$$\text{nnz_K_col}, \text{K_density_col} = \frac{\text{nnz}(K)}{\text{total entries}}, \text{K_bandwidth_col},$$

目的就是檢查 sparse pattern 與 bandwidth。從 summary table 可看出，當網格由 6×2 加密到 96×32 時，自由度由 42 增加到 6402，但矩陣密度從 2.9478×10^{-1} 下降到 2.7151×10^{-3} 。這代表矩陣規模雖然快速變大，但非零元素比例反而快速下降。這是 FEM 重要的數值結構，也是之後選擇 sparse solver 的理由。

(v) 條件數：網格加密後 K_{ff} 會愈來愈 ill-conditioned

即使 K_{ff} 是 SPD，網格加密後 condition number 仍會上升。這是 elliptic-type

stiffness operator 離散化後的現象；當 mesh size h 變小，自由度增加，最小與最大特徵尺度差距會變大，矩陣對誤差與 round-off 的敏感度也會提高。程式用

condest_Kfree_col 估計 K_{ff} 的 condition number，正是為了觀察這件事。

這個現象說明：知道矩陣是 SPD 還不夠，還要用合適的矩陣排序、稀疏分解或預條件化迭代法。否則網格更細時，即使理論上有唯一解，實際計算效率與穩定性仍可能變差。

(vi) 結論

assembled stiffness matrix K 的特徵可以整理為

- 施加邊界條件前， K 是 sparse, symmetric, and positive semidefinite
- 施加固定端 essential boundary condition 並取出自由自由度後，真正要求解

的矩陣 K_{ff} 是 sparse, symmetric, and positive definite

(2)

根據第 (1) 小題，施加固定端邊界條件後，實際需求解的是 $K_{ff}U_f = F_f$ ，其中

K_{ff} 是 sparse SPD matrix。因此，解法的選擇應該要充分利用 sparse、

symmetric、positive definite 這三個特性。

(i) 為何不選擇求反矩陣，或 Gaussian elimination

數學上可以寫成 $U_f = K_{ff}^{-1}F_f$ ，但在數值線性代數中，求 inverse 會增加運算

量、破壞稀疏性，而且可能放大 rounding error。若把 FEM 的 sparse matrix 轉成 dense matrix 再做一般 Gaussian elimination，也會浪費大量記憶體與時間；自由度稍微增加後，計算成本就會變得不合理。

(ii) 中等規模問題：使用 sparse Cholesky factorization

對 SPD matrix，最自然的 solver 是 Cholesky factorization。採用上三角形，可寫為 $K_{ff} = R^T R$ ，解線性系統時，不需求反矩陣，只要做兩次三角系統

(Forward/Backward substitution) 求解： $R^T y = F_f$ ， $RU_f = y$ ，這比一般 LU

decomposition 更適合，因為 Cholesky 直接利用 SPD 結構，只需處理約一半的矩陣資訊；若搭配 sparse matrix ordering，還能簡化 factorization 過程。

程式的主設定為 solverType = 'chol'。在 solveLinearSystem 中，程式先使用

```
424         switch lower(solverType)
425         case 'chol'
426             p = symamd(A);
427             Ap = A(p, p);
428             bp = b(p);
```

對 sparse symmetric matrix 做重排，再對重新排序後的矩陣執行

```
[R, cholFlag] = chol(Ap, 'upper');
solverInfo.chol_flag = cholFlag;

if cholFlag == 0
    xp = R \ (R.' \ bp);
```

symamd 的不改變方程本身，而是重新排列自由度順序，使 Cholesky 分解時產生較少 fill-in，進而節省記憶體與運算時間。求得 reordered solution 之後，程式再還原回原本的 global DOF ordering。

(iii) 大規模問題：使用 CG/PCG 與 incomplete Cholesky preconditioner

若網格進一步加密，直接 Cholesky 的記憶體需求可能快速增加。這時更適合使用 conjugate gradient method (CG) 或 preconditioned conjugate gradient method (PCG)。CG 是專門為 SPD linear system 設計的 Krylov subspace iterative method；它不需要完整分解 K_{ff} ，主要計算成本是 sparse matrix-vector

multiplication，因此很適合大型 FEM 問題。而停止條件可寫為

$$\frac{\|r^{(k)}\|_2}{\|b\|_2} = \frac{\|b - K_{ff}U_f^{(k)}\|_2}{\|b\|_2} < \varepsilon。 \text{ 程式在 Cholesky 失敗時會自動 fallback 到 pcg，並}$$

先嘗試 incomplete Cholesky preconditioner：

```
451 try
452     opts.type = 'ict';
453     opts.droptol = 1.0e-3;
454     opts.diagcomp = 1.0e-8;
455     L = ichol(A, opts);
456     [x, flag, relres, iter] = pcg(A, b, tol, maxit, L, L.');
```

(iv)數值檢查

程式額外記錄 solver_flag_col、solver_relres_col、solver_iter_col 與 chol_flag_free_col。這些欄位可以檢查線性系統是否真正被解出來。實際結果中，Cholesky flag 皆為 0，solver relative residual 約落在 10^{-13} 到 10^{-12} 的量級，表示離散線性系統求解誤差遠小於主要的離散化誤差。

(v)結論

在 SPD 結構成立且規模可承受時用 sparse Cholesky；若直接分解不順利，則改用具有預條件的 PCG。

(3)

FEM 必須先定義誤差是用哪一種 norm 衡量。位移誤差、應變誤差、應力誤差與 energy norm error 的收斂階數不同。

(i)Q4 bilinear element 的 approximation order

此題使用的是 Q4 bilinear quadrilateral element。形狀函數在自然座標 ξ_1, ξ_2 的每個方向都是一次，因此 polynomial degree 可視為 $p = 1$ 。對夠平滑的解析解，標準 FEM 誤差估計為 $\|u - u_h\|_{H^1(\Omega)} = O(h^p) = O(h)$ (H^1 為 Sobolev norm)，及

$\|u - u_h\|_{L^2(\Omega)} = O(h^{p+1}) = O(h^2)$ ，這裡 $h = \max\{\Delta x_1, \Delta x_2\}$ 是 mesh size。(u 為精確

解， u_h 為 FEM 數值解) $(\|u - u_h\|_{H^1(\Omega)} = \left(\int_{\Omega} |u - u_h|^2 d\Omega + \int_{\Omega} |\nabla u - \nabla u_h|^2 d\Omega \right)^{1/2} \Rightarrow$

同時看位移誤差以及位移梯度誤差)

說明:

對 Q4 bilinear element 而言，位移在每個元素內由一次形狀函數近似，因此 polynomial degree 可視為 $p = 1$ ，若精確解足夠平滑，標準有限元素插值誤差估計

計給出 $\|u - I_h u\|_{L^2(\Omega)} \leq Ch^{p+1} |u|_{H^{p+1}(\Omega)}$ ，以及 $|u - I_h u|_{H^1(\Omega)} \leq Ch^p |u|_{H^{p+1}(\Omega)}$ ，

其中 $I_h u$ 是將精確解插值到有限元素空間中的近似，C 為與 h 無關的常數。由

Galerkin finite element method 的最佳近似性，有限元素解 u_h 的誤差階數會與此

插值誤差同階。因此代入 $p=1$ 可得 $|u-u_h|_{H^1(\Omega)}=O(h)$, 而 displacement 的 L^2 error

預期為 $\|u-u_h\|_{L^2(\Omega)}=O(h^2)$ 。這兩個結果的直觀意義是：位移場本身以 Q4 元素

近似時，理想情況下可達到二階收斂；但 strain 與 stress 需要由位移對空間微分得到，微分會使近似階數下降一階，因此 strain/stress 或 energy norm 相關誤差通常只預期一階收斂。

結論：

- displacement L^2 error 預期為 $O(h^2)$

- strain/stress/energy error 預期為 $O(h)$

(ii) 看 displacement error，預期二階收斂

若誤差定義為 displacement 的 L^2 norm，例如 $e_u(h) = \frac{\left(\int_{\Omega} \|u_h - u_{\text{exact}}\|_2^2 d\Omega\right)^{1/2}}{\left(\int_{\Omega} \|u_{\text{exact}}\|_2^2 d\Omega\right)^{1/2}}$ ，則 Q4

bilinear element 的理想預期是 $e_u(h) = O(h^2)$ 。正是程式中的

relative_L2_displacement_error_col 所衡量的量。程式也計算相對於最細網格解

的 RMS error： $e_{\text{ref}}(h) = \frac{\left(\frac{1}{N} \sum_i \|u_h(x_i) - u_{\text{ref}}(x_i)\|_2^2\right)^{1/2}}{\left(\frac{1}{N} \sum_i \|u_{\text{ref}}(x_i)\|_2^2\right)^{1/2}}$ 。這個量的目的是提供另一個

self-convergence 檢查。

(iii) 看 strain、stress 或 energy norm，預期一階收斂

應變是位移的一階導數： $\epsilon = \nabla_s u$ ，應力又由 $\sigma = D\epsilon$ 得到。因此 strain/stress error

本質上與 H^1 seminorm error 同階。對 Q4 element 而言，預期為

$\|\epsilon - \epsilon_h\| = O(h)$, $\|\sigma - \sigma_h\| = O(h)$ 。

(iv) 觀測 convergence rate 的估計方式

若一組網格大小為 h_i ，對應誤差為 e_i ，則相鄰兩組網格的 observed order 可估為

$r_i = \frac{\log(e_{i-1}/e_i)}{\log(h_{i-1}/h_i)}$ ，程式中的 rate_exact_relative_L2 依此公式計算 displacement

relative L^2 error 的觀測 convergence rate；rate_reference_relative_rms 則用最細網格當 reference solution 做同樣估計。

程式使用 $(N_x, N_y) = (6, 2), (12, 4), (24, 8), (48, 16), (96, 32)$ ，每次將 h 約略減半，因

此很適合用 log-log slope 觀察 spatial convergence rate。

(v) 結論

Q4 bilinear element 的 polynomial degree 為 $p = 1$ 。若以 displacement relative L^2 error 或 nodal RMS displacement error 衡量，預期 spatial convergence rate 為

$O(h^2)$ 。若以 strain、stress 或 energy norm 衡量，因為它們涉及位移的一階導數，預期 spatial convergence rate 為 $O(h)$ 。

(4) 程式架構、數值設定與驗證

(i) 網格設定

程式使用五組網格： $(N_x, N_y) = (6, 2), (12, 4), (24, 8), (48, 16), (96, 32)$ ，其中 N_x 是 x_1

方向元素數， N_y 是 x_2 方向元素數。網格間距為 $h_x = \frac{L}{N_x}$ ， $h_y = \frac{2a}{N_y}$ ，並定義

$h = \max(h_x, h_y)$ 。設定 $\frac{L}{N_x} = \frac{2a}{N_y}$ ，所以每次網格加密時元素大致保持正方形，避

免了元素長寬比過大造成的數值誤差。

(ii) 程式流程

整體 MATLAB 程式流程如下：

- 建立矩形 Q4 mesh：節點座標由 createRectMesh 產生，元素採 counter-clockwise ordering。
- 對每個元素計算 K^e ：在四個 Gauss points 上計算 J 、 $\det(J)$ 、 B ，再累加 $bB^T DB \det(J)$ 。
- 組裝全域矩陣 K ：利用 global DOF indices 將 8×8 的 K^e 放入 sparse matrix。
- 施加外力：

■ 舊版：在左端中心點 $(0, 0)$ 的 u_2 自由度施加 -8000N 。

■ 新版：於自由端面 $x_1 = 0$ 上施加均勻分佈載重。程式中每個自由端邊界元素的兩個端點各加入 $\frac{P_s}{2N_y}$ 的垂直力，因此整條自由端邊界的總垂直

$$\text{外力滿足 } \sum_i F_{2,i} = P_s = -8000\text{N}$$

- 施加固定端：固定所有 $x_1 = L$ 節點的 u_1, u_2 。
- 求解線性系統：以 sparse Cholesky 為主，失敗時 fallback 到 PCG。
- 後處理：計算節點位移、元素中心應力、free-end displacement、relative

errors、convergence rates，並輸出圖檔與 CSV 表格。

(iii)驗證量與輸出檢查

- 總外力檢查：
 - 舊版:total_F2_N_col 每組網格皆為-8000N，代表集中力施加正確。
 - 新版: $\sum_i F_{2,i} = -8000\text{N}$
- 矩陣對稱性檢查：K_symmetry_relative_error_col 為 0，代表 assembly 後 K 保持對稱。
- Cholesky 檢查：chol_flag_free_col 皆為 0，代表 K_{ff} 可成功 Cholesky 分解。
- 殘差檢查：solver relative residual 約為 10^{-13} 至 10^{-12} ，表示線性系統解得準。
- 稀疏性檢查：matrix density 隨網格加密快速下降，符合 FEM sparse structure。
- 收斂性檢查：對多組 h 計算 exact-field error 與 finest-mesh reference error，觀察 log--log slope。

(iv)數值結果表與定量分析

下表摘錄自 MATLAB summary table 的主要數據(結合新舊程式 error)。

Table 3 (舊版 point load)

Case	$N_x \times N_y$	DOFs	h [mm]	nnz(K)	density(K)	$u_2(0,0)$ [mm]	e_{exact}	e_{ref}
1	6×2	42	10.000	520	2.9478×10^{-1}	-8.2234	1.4829×10^{-1}	1.2340×10^{-1}
2	12×4	130	5.000	1902	1.1254×10^{-1}	-9.0371	6.8823×10^{-2}	3.6488×10^{-2}
3	24×8	450	2.500	7224	3.5674×10^{-2}	-9.3075	4.5560×10^{-2}	9.7359×10^{-3}
4	48×16	1666	1.250	28192	1.0157×10^{-2}	-9.4085	3.9320×10^{-2}	2.1525×10^{-3}
5	96×32	6402	0.625	111278	2.7151×10^{-3}	-9.4621	3.7591×10^{-2}	0

Table 4 (新版 uniform load)

Case	$N_x \times N_y$	DOFs	h [mm]	$u_2(0,0)$ [mm]	e_{exact}	e_{ref}
1	6×2	42	10.000	-8.1981	1.4821×10^{-1}	1.2263×10^{-1}
2	12×4	130	5.000	-8.9855	6.8571×10^{-2}	3.6225×10^{-2}
3	24×8	450	2.500	-9.2237	4.5257×10^{-2}	9.6586×10^{-3}
4	48×16	1666	1.250	-9.2901	3.9007×10^{-2}	2.1346×10^{-3}
5	96×32	6402	0.625	-9.3087	3.7276×10^{-2}	0

由 **Table 3** 中可以看出幾件事。第一，隨著網格加密，自由度從 42 增加到 6402，但矩陣密度從 2.9478×10^{-1} 降到 2.7151×10^{-3} 。這是 FEM 稀疏矩陣特性的數值證據。第二，free-end displacement 從 -8.2234mm 逐漸接近 -9.4621mm，方向與大小都合理。第三，對舊版題目而言 exact-field error 後期趨於平緩，而 reference error 下降接近 2 階，這支持前面理論。

可以看出，兩種載重模型的整體趨勢非常接近。因為兩者使用相同的總垂直外力 $\sum_i F_{2,i} = -8000\text{N}$ ，且同一根懸臂梁的 global bending response 主要由總力、梁

長、截面剛度與固定端條件控制。因此，無論將外力集中在自由端中心節點，或是均勻分佈於自由端面，只要總力相同，整體撓曲行為會非常接近。

不過兩者仍然存在細微的差異。以自由端中心位移 $u_2(0,0)$ 來看，uniform-load model 的位移量略小於 point-center load model。例如在最細網格 96×32 下，

$u_2^{\text{point}}(0,0) = -9.4621\text{mm}$, $u_2^{\text{uniform}}(0,0) = -9.3087\text{mm}$ ，兩者差距

$|u_2^{\text{point}} - u_2^{\text{uniform}}| = 0.1534\text{mm}$ ，相對於解析場自由端位移 $|u_{2,\text{exact}}(0,0)| = 9.5760\text{mm}$

此差異約為 $\frac{0.1534}{9.5760} \approx 1.60\%$ 。這說明在位移上，兩種載重模型的差異不大；但

uniform load 因為把外力沿端面分散，局部載重集中程度較低，因此自由端中心位移略小。point-center load 比 uniform load 稍微更容易在中心附近造成局部柔化與較大的局部撓度。

(v) Observed convergence order

在誤差方面，兩種模型的 e_{exact} 都隨網格加密下降，但後期下降趨緩。這表示直接和 Timoshenko--Goodier exact field 比較時，誤差中不只包含有限元素離散誤差，也包含 FEM 邊界載重表示與解析梁解之間的模型差異。而 e_{ref} 是與最細網格解比較的 self-convergence error。從表中可見，兩種模型的 e_{ref} 幾乎同步下降，且 uniform-load model 的誤差略低一些。新版 uniform-load 由相鄰網格估計的 observed order 約為 1.76, 1.91, 2.18, 整體接近二階。這和 Q4 bilinear element 對 displacement field 的理論預期相符。

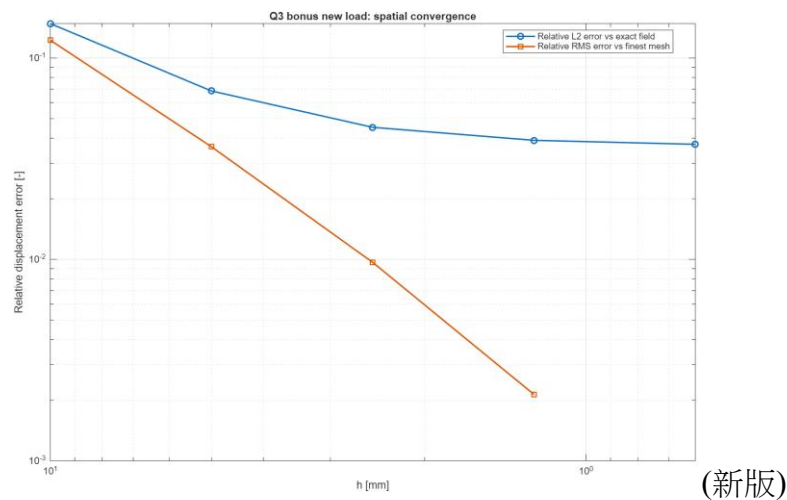
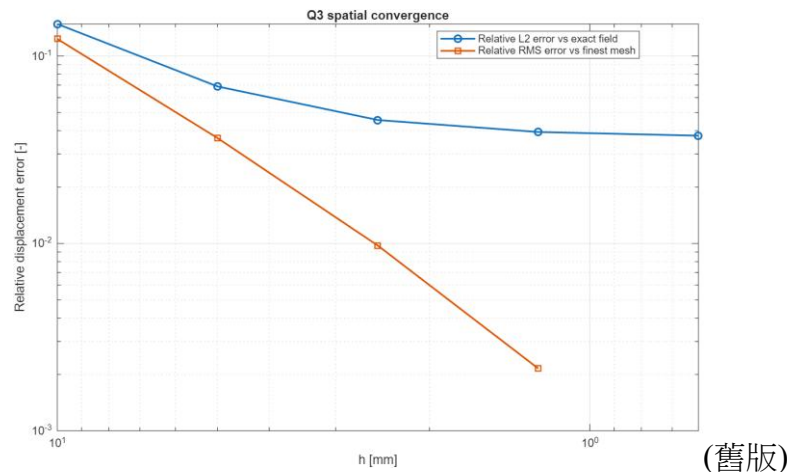
程式以 $p_{\text{obs}} = \frac{\log(e_{i-1}/e_i)}{\log(h_{i-1}/h_i)}$ 估計 observed order。對舊版 reference RMS error，得到

1.758, 1.906, 2.177。這三個值整體非常接近 2。因此，若以同一 point-load FEM model 的 self-convergence 觀察位移場，結果符合 Q4 bilinear element 的二階 displacement convergence 預期。

相較之下，舊版 exact-field error 的 observed order 由 1.108 降到 0.595、0.213、0.065，後期不再呈現單純離散誤差收斂。這不代表 FEM 失敗，是因為程式採用的是單一節點集中力模型；但 Timoshenko--Goodier 解析場對應的是較平滑的端面應力分布。兩者雖然具有相同的總外力與相近的整體彎曲行為，但在受力端附近的邊界資料並不完全相同。因此，當網格加密後，FEM 離散誤差逐漸下降，剩下的差異主要來自模型本身的邊界設定差異，使得 exact-field error curve 後期趨於平緩。

(vi) 輸出圖形結果

(vi)-1 Spatial convergence



▲ Fig.6 空間收斂性。藍線表示 FEM 解與解析場的比較；橘線表示各網格解與最細網格解的比較

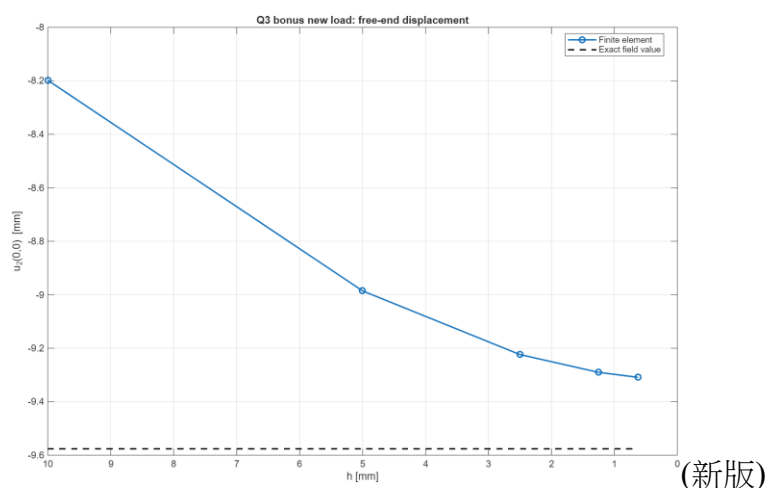
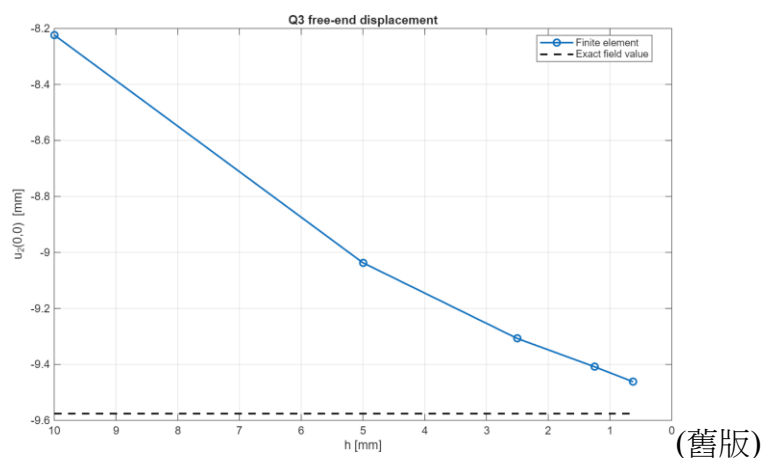
新版:藍線為 FEM 解與題目給定 Timoshenko--Goodier exact field 的 relative L^2 displacement error，橘線則為各網格解相對於最細網格解的 relative RMS error。由圖中可見，橘線在 log--log 圖上呈現接近直線下降，表示同一個 uniform-load FEM model 在網格加密下具有穩定的 self-convergence。由於本題使用的是 Q4 bilinear element，位移場在 L^2 或 RMS 下理論上預期為 $\|\mathbf{u} - \mathbf{u}_h\|_{L^2} = O(h^2)$ 橘線接近二階下降，支持 Q4 element 對 displacement field 的理論收斂預期。藍線雖然也隨網格加密下降，但後期下降速度較慢，並未像橘線一樣維持完整二階。原因是藍線比較的是 FEM uniform-load model 與題目給定的解析場；解析場是 Timoshenko--Goodier beam solution，而 FEM 中實際施加的是有限元素邊界上的 constant traction consistent load。兩者在整體力與彎曲行為上相近，但在端面局部 traction distribution 與二維邊界條件表示上仍可能存在差異。因此，藍線可視為 FEM 解與解析場的參考比較；橘線則更適合用來判斷同一數值模型本身的 mesh convergence。

舊版:藍線一開始下降，但在 h 很小時趨於平緩，最後約停在 3.76×10^{-2} 。實際上這條線混入了離散誤差與 point-load FEM model 與 smooth exact field 之間的局部模型差異。當網格夠細時，離散誤差下降，模型差異成為主要項，因此藍線可能會趨於平緩。

橘線則以最細網格作 reference，衡量同一個離散模型下的 mesh refinement behavior。它在 log-log 圖上接近直線，斜率約為 2。這表示位移場本身確實呈現接近二階的 self-convergence。對 Q4 bilinear element 而言，這個結果合理。

小結：exact-field error 主要反映局部模型差異；self-convergence error 則驗證了位移場接近二階收斂。

(vi)-2 Free-end displacement



▲ Fig.7 網格加密下自由端垂直位移 $u_2(0,0)$ 的變化

隨著 h 逐漸減小，FEM 結果單調地往 exact field value 靠近，表示整體彎曲柔度與固定端邊界條件皆被合理捕捉。

和舊版 point-center load 的結果相比，新版 uniform load 的自由端位移趨勢非常接近，但數值上不相同。因為兩者雖然總外力皆為 -8000N ，但外力在自由端面的分佈方式不同。point-center load 將所有外力集中於中心節點，uniform load 則把外力沿整個自由端面分散到邊界節點。對細長梁而言，整體彎曲位移主要

由總外力與整體彎矩控制，因此兩種模型的 global deformation 會很相似；但自由端附近的局部位移與應力仍會受到載重分佈方式影響。

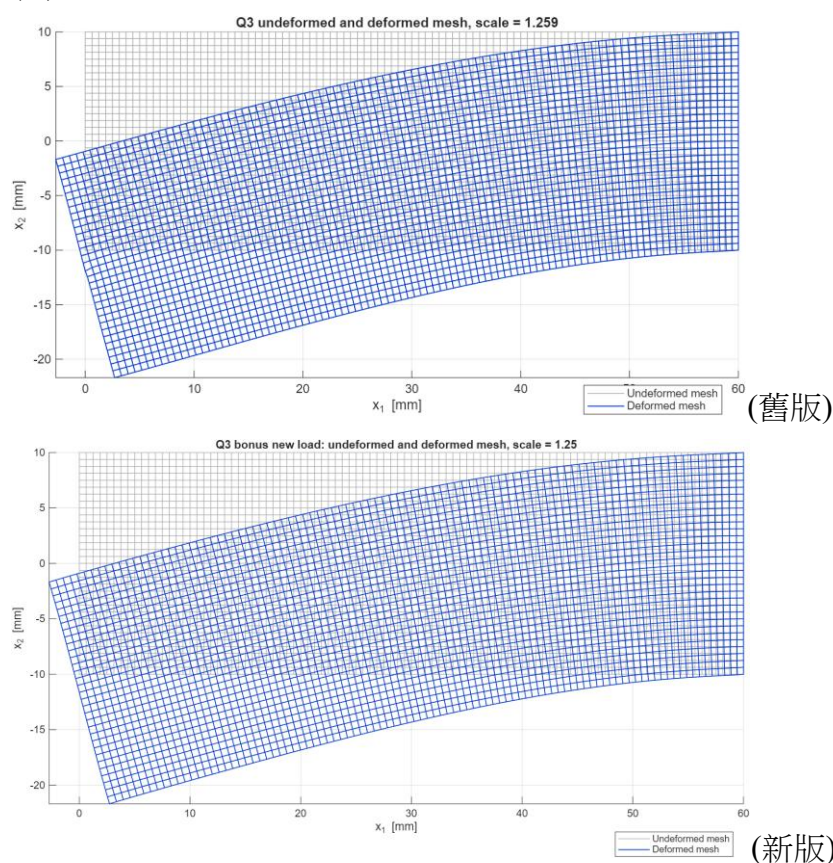
舊版解析場給出 $u_{2,\text{exact}}(0,0) = -9.576\text{mm}$ ，FEM 結果隨網格加密為

$-8.2234, -9.0371, -9.3075, -9.4085, -9.4621\text{ mm}$ 。隨著 h 由 10mm 降到 0.625mm ，FEM 位移單調接近 exact field value。最細網格的 displacement error 約為

$|-9.4621 - (-9.5760)| = 0.1139\text{mm}$ ，相對於解析值大小，約為 $\frac{0.1139}{9.5760} \approx 1.19\%$ 。

這說明即使使用 point load，FEM 已能相當準確地捕捉整體柔度與端點撓度。

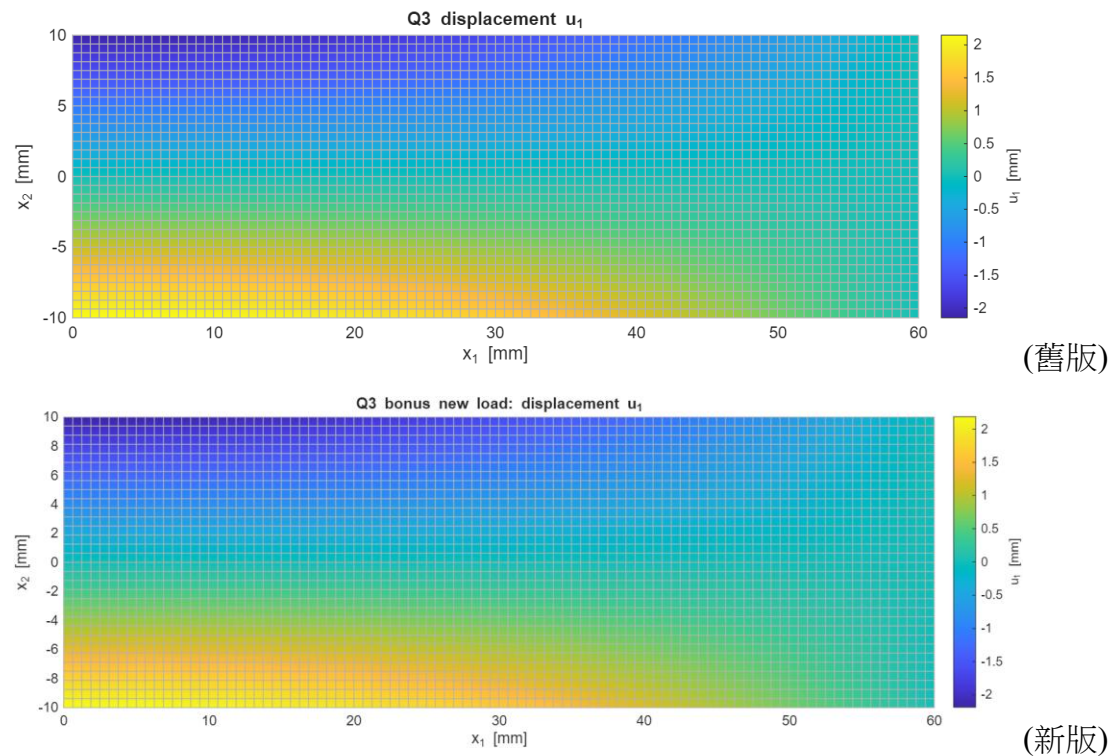
(vi)-3 Undeformed and deformed mesh



▲ Fig.8 Undeformed and deformed mesh

圖 8 比較未變形網格與放大後(以便更清楚看出變化)的變形網格。右端固定、左端受向下力後，整根梁呈現典型懸臂梁彎曲形狀。圖中 deformation scale 約為 1.259。

(vi)-4 Displacement field u_1



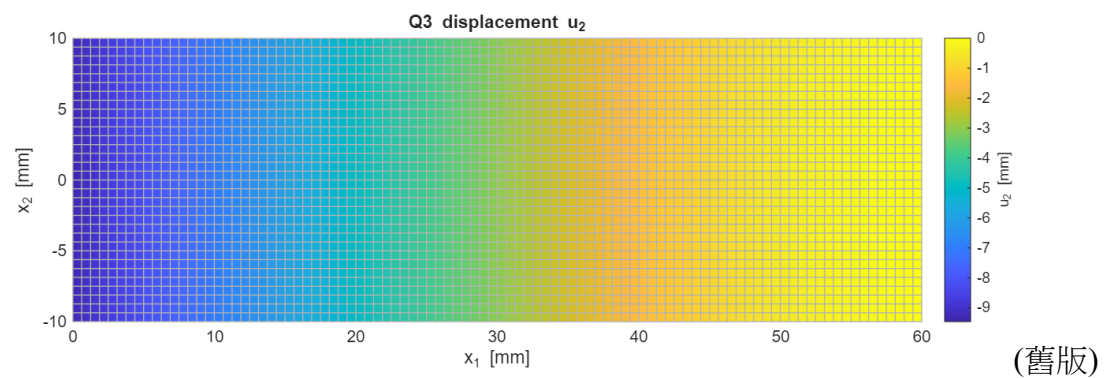
▲ Fig.9 Horizontal displacement field u_1

圖 9 顯示水平位移 u_1 。可以看到 u_1 在梁的上半部與下半部呈現相反符號，且中性軸附近接近零。這正是 **bending** 的典型特徵：梁受彎時，上下會產生相反方向的軸向伸縮與位移。

解析應力給出 $\sigma_1 = -\frac{P_s x_1 x_2}{I}$ ，因此 σ_1 與 x_2 成正比並在 $x_2 = 0$ 換號。既然軸向應力

與軸向應變有關，而 $\epsilon_1 = \partial u_1 / \partial x_1$ ，則 u_1 在上下半部呈現不同符號。

(vi)-5 Displacement field u_2



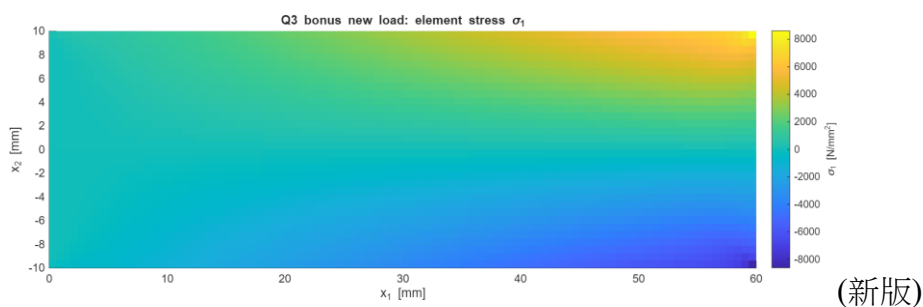
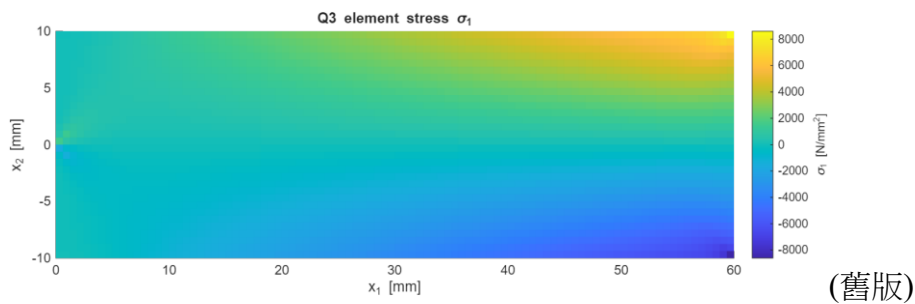


▲ Fig.10 Vertical displacement field u_2

圖 10 顯示垂直位移 u_2 。整個梁大多為負值，代表結構向下撓曲；固定端附近接近零，自由端附近達到最大負位移。這與圖 7 的 free-end displacement 結果一致。

u_2 沿 x_1 方向有主要變化，沿 x_2 方向變化較平滑，這符合 Euler-Bernoulli / Timoshenko 梁彎曲特性：截面大致一起下移，撓度主要沿梁長方向累積。固定端整條邊 $u_2 \approx 0$ ，也確認固定端 BC 條件處理正確。

(vi)-6 Normal stress σ_1



▲ Fig.11 Element stress σ_1 at element centers

圖 11 顯示元素中心的 σ_1 。整體分布呈現典型彎曲應力：中性軸附近接近零，

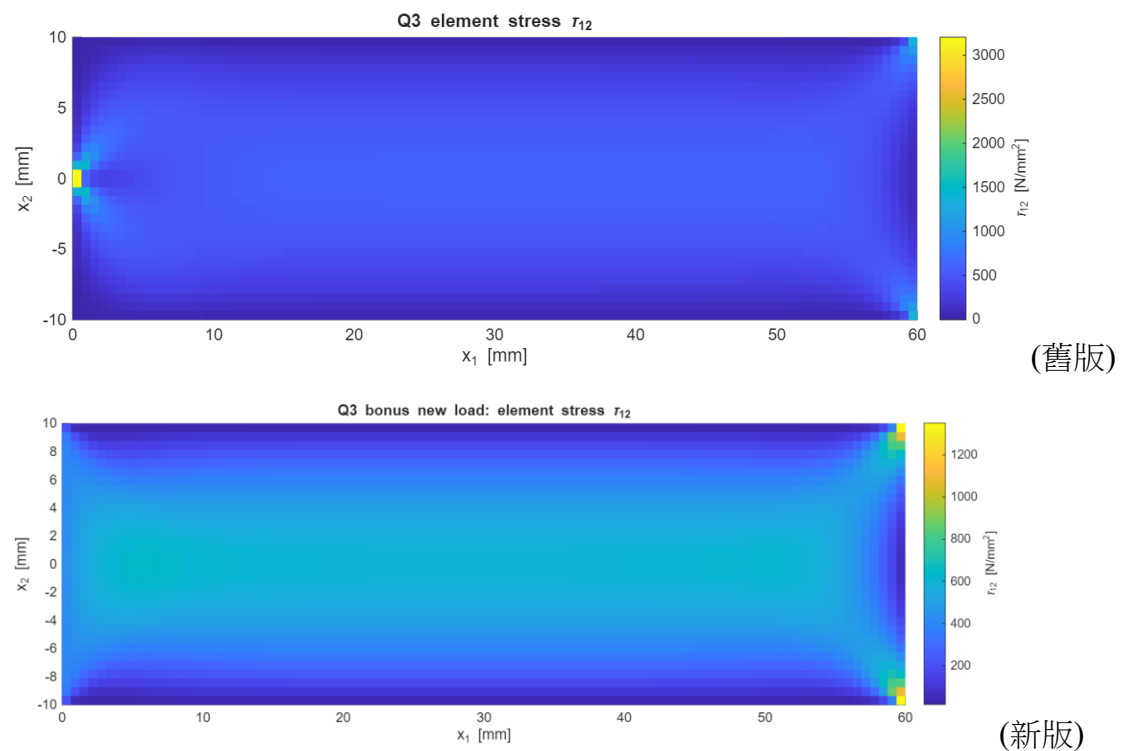
上下表面符號相反，且靠近固定端時應力絕對值較大。這和解析式 $\sigma_1 = -\frac{P_s x_1 x_2}{I}$

相符。因為 $|x_1|$ 越靠近固定端越大，彎矩越大，所以 σ_1 在固定端附近最大；因

為 σ_1 與 x_2 成正比，所以上下緣應力最大、中性軸為零。

舊版左端受力點附近可看到局部擾動，這是集中力造成的局部應力集中。

(vi)-7 Shear stress τ_{12}



▲ Fig.12 Element shear stress τ_{12} at element centers

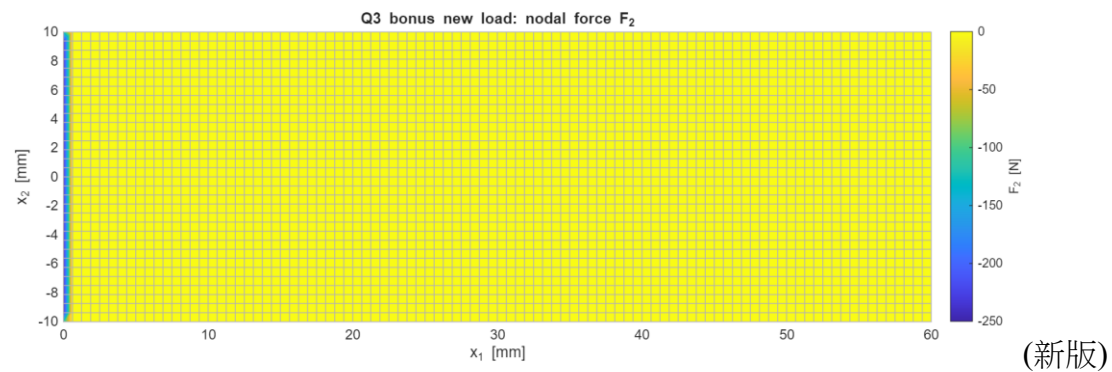
圖 12 顯示剪應力 τ_{12} 。解析式為 $\tau_{12} = -\frac{P_s}{2I}(a^2 - x_2^2)$ ，由於程式使用

$P_s = -8000\text{N}$ ，所以 τ_{12} 在截面內多為正值，且在 $x_2 = 0$ 附近較大、在上下表面

$x_2 = \pm a$ 附近較小。這是矩形截面梁剪應力的拋物線分布特徵。

與舊版 point-center load 相比，新版 uniform load 的 τ_{12} 場較平滑，尤其是自由端中心附近不再出現由單一節點集中力造成的強烈局部尖峰。舊版將全部外力集中在中心節點，因此自由端中心附近會有較明顯的應力集中；新版則把外力沿自由端面分散到邊界節點，使局部剪應力分布更接近連續載重的物理情形。

(vi)-8 nodal force



▲ Fig.13 nodal force F_2

新版程式新增的 F_2 nodal force contour 是為了檢查 uniform-load boundary setting。圖中只有自由端面 $x_1 = 0$ 上的節點具有非零垂直外力，其餘內部節點與固定端節點外力皆為零。符合新版題目。

程式中對每個自由端邊界元素，將其邊界總力平均分配給上下兩個端點：

$$F_{2,\text{end}} = \frac{P_s}{2N_y}$$

由於內部邊界節點同時屬於上下兩個邊界元素，所以內部自由端節點會累積兩次；自由端上下角點只屬於一個邊界元素，因此角點力為內部邊界節點的一半。

Appendix

本作業之 MATLAB 程式碼撰寫過程中，Q2、Q3 有使用 AI 作為輔助工具
附上 AI 對話連結。

Q2 AI link: <https://chatgpt.com/share/6a19f240-b760-8322-a68e-b06ec29b6e24>

Prompt: 我正在完成本課程期末專題第二題。根據課程規範，AI 只能協助撰寫
或除錯對應的程式碼，不能協助撰寫報告內容、進行結果分析、解釋物理意
義、翻譯或潤飾文字。

請你只協助我撰寫第二題所需的程式碼。

使用的程式語言：

MATLAB

1. 請只提供完成計算任務所需的程式碼。
2. 請不要撰寫任何報告段落。
3. 請不要分析或解釋數值結果。
4. 請不要提供討論、結論、物理意義或科學寫作內容。
5. 請不要翻譯或潤飾任何報告文字。
6. 可以在程式碼中加入簡潔註解，用來說明程式邏輯。
7. 程式碼需要清楚、可重現、容易修改。
8. 圖形需要包含清楚的座標軸標籤、圖例、單位與標題；若合適，請加入儲存圖檔或輸出資料的指令。

Q2 code:

```
clear; close all; clc;

format long e

N_grid_list = [11 21 41 81 161 321];

outDir = 'q2_outputs_revised';

saveFigures = true;

if ~exist(outDir, 'dir')

    mkdir(outDir);

end
```

```
exact_y = @(x) (0.25 + (2*exp(2) - 0.25).*x).*exp(-2.*x) + 0.25.*(x - 1);
```

```
Delta_x_values = 1 ./ (N_grid_list - 1);
```

```
maxErrFD = zeros(numel(N_grid_list), 1);
```

```
maxErrFE = zeros(numel(N_grid_list), 1);
```

```
maxErrCN = zeros(numel(N_grid_list), 1);
```

```
methodName = cell(0, 1);
```

```
N_grid_col = [];
```

```
Delta_x_col = [];
```

```
maxAbsError_col = [];
```

```
discreteL2Error_col = [];
```

```
rmsError_col = [];
```

```
endpointError_col = [];
```

```
theta_col = [];
```

```
row = 0;
```

```
for idx = 1:numel(N_grid_list)
```

```
    N_grid = N_grid_list(idx);
```

```
    Delta_x = 1 / (N_grid - 1);
```

```
    [x_fd, y_fd, e_fd] = finite_difference_solution(N_grid, exact_y);
```

```
    maxErrFD(idx) = max(abs(e_fd));
```

```

row = row + 1;

methodName{row, 1} = 'Finite difference';

N_grid_col(row, 1) = N_grid;

Delta_x_col(row, 1) = Delta_x;

maxAbsError_col(row, 1) = max(abs(e_fd));

discreteL2Error_col(row, 1) = sqrt(Delta_x * sum(e_fd.^2));

rmsError_col(row, 1) = sqrt(mean(e_fd.^2));

endpointError_col(row, 1) = max(abs([y_fd(1) - 0; y_fd(end) - 2]));

theta_col(row, 1) = NaN;


[x_fe, y_fe, e_fe, theta_fe] = shooting_solution(N_grid, 'ForwardEuler', exact_y);

maxErrFE(idx) = max(abs(e_fe));


row = row + 1;

methodName{row, 1} = 'Shooting Forward Euler';

N_grid_col(row, 1) = N_grid;

Delta_x_col(row, 1) = Delta_x;

maxAbsError_col(row, 1) = max(abs(e_fe));

discreteL2Error_col(row, 1)= sqrt(Delta_x * sum(e_fe.^2));

rmsError_col(row, 1) = sqrt(mean(e_fe.^2));

endpointError_col(row, 1) = max(abs([y_fe(1) - 0; y_fe(end) - 2]));

theta_col(row, 1) = theta_fe;


[x_cn, y_cn, e_cn, theta_cn] = shooting_solution(N_grid, 'CrankNicolson',
exact_y);

```

```

maxErrCN(idx) = max(abs(e_cn));

row = row + 1;

methodName{row, 1} = 'Shooting Crank-Nicolson';

N_grid_col(row, 1) = N_grid;

Delta_x_col(row, 1) = Delta_x;

maxAbsError_col(row, 1) = max(abs(e_cn));

discreteL2Error_col(row, 1) = sqrt(Delta_x * sum(e_cn.^2));

rmsError_col(row, 1) = sqrt(mean(e_cn.^2));

endpointError_col(row, 1) = max(abs([y_cn(1) - 0; y_cn(end) - 2]));

theta_col(row, 1) = theta_cn;

end

error_summary = table(methodName, N_grid_col, Delta_x_col, maxAbsError_col,
discreteL2Error_col, rmsError_col, endpointError_col, theta_col, ...

    'variableNames', {'method', 'N_grid', 'Delta_x', 'max_abs_error',
'discrete_L2_error', 'rms_error', 'endpoint_error', 'theta'});

disp(error_summary)

writetable(error_summary, fullfile(outDir, 'q2_error_summary_revised.csv'));

orderFD = observed_order(Delta_x_values, maxErrFD.');
orderFE = observed_order(Delta_x_values, maxErrFE.');
orderCN = observed_order(Delta_x_values, maxErrCN.');

```

```

order_summary = table(N_grid_list(2:end).', Delta_x_values(2:end).', orderFD.',
orderFE.', orderCN.', ...

    'variableNames', {'N_grid', 'Delta_x', 'order_FD', 'order_shoot_FE',
'order_shoot_CN'}));

disp(order_summary)

writetable(order_summary, fullfile(outDir, 'q2_order_summary_revised.csv'));

N_plot = N_grid_list(end);

[x_plot, y_fd_plot, e_fd_plot] = finite_difference_solution(N_plot, exact_y);

[~, y_fe_plot, e_fe_plot] = shooting_solution(N_plot, 'ForwardEuler', exact_y);

[~, y_cn_plot, e_cn_plot] = shooting_solution(N_plot, 'CrankNicolson', exact_y);

y_exact_plot = exact_y(x_plot);

solution_table = table(x_plot, y_exact_plot, y_fd_plot, y_fe_plot, y_cn_plot, ...

    abs(e_fd_plot), abs(e_fe_plot), abs(e_cn_plot), ...

    'variableNames', {'x', 'y_exact', 'y_FD', 'y_shoot_FE', 'y_shoot_CN', ...

    'abs_error_FD', 'abs_error_shoot_FE', 'abs_error_shoot_CN'}));

writetable(solution_table, fullfile(outDir, 'q2_solution_finetest_grid_revised.csv'));

figure('Color', 'w');

plot(x_plot, y_exact_plot, 'Linewidth', 2); hold on;

plot(x_plot, y_fd_plot, 'o', 'MarkerSize', 4);

plot(x_plot, y_fe_plot, 's', 'MarkerSize', 4);

```

```

plot(x_plot, y_cn_plot, '^', 'MarkerSize', 4);

grid on;

xlabel('x [-]');

ylabel('y [-]');

title(sprintf('Q2 solution comparison, N = %d, \\Delta x = 1/%d', N_plot, N_plot -
1));

legend('Exact', 'Finite difference', 'Shooting: Forward Euler', 'Shooting: Crank-
Nicolson', 'Location', 'best');

save_current_figure(outDir, 'q2_solution_comparison_revised', saveFigures);

figure('Color', 'w');

semilogy(x_plot, max(abs(e_fd_plot), eps), 'o-', 'MarkerSize', 4); hold on;

semilogy(x_plot, max(abs(e_fe_plot), eps), 's-', 'MarkerSize', 4);

semilogy(x_plot, max(abs(e_cn_plot), eps), '^-', 'MarkerSize', 4);

grid on;

xlabel('x [-]');

ylabel('Absolute error [-]');

title(sprintf('Q2 absolute error distribution, N = %d, \\Delta x = 1/%d', N_plot,
N_plot - 1));

legend('Finite difference', 'Shooting: Forward Euler', 'Shooting: Crank-Nicolson',
'Location', 'best');

save_current_figure(outDir, 'q2_error_distribution_revised', saveFigures);

figure('Color', 'w');

loglog(Delta_x_values, maxErrFD, 'o-', 'MarkerSize', 6); hold on;

```

```

loglog(Delta_x_values, maxErrFE, 's-', 'MarkerSize', 6);

loglog(Delta_x_values, maxErrCN, '^-', 'MarkerSize', 6);

set(gca, 'xDir', 'reverse');

grid on;

xlabel('Grid spacing \Delta x [-]');

ylabel('Maximum absolute error [-]');

title('Q2 grid convergence');

legend('Finite difference', 'Shooting: Forward Euler', 'Shooting: Crank-Nicolson',

'Location', 'best');

save_current_figure(outDir, 'q2_grid_convergence_revised', saveFigures);

save(fullfile(outDir, 'q2_workspace_results_revised.mat'), ...

    'error_summary', 'order_summary', 'solution_table', 'N_grid_list',

'Delta_x_values', ...

    'maxErrFD', 'maxErrFE', 'maxErrCN');

function [x, y, e] = finite_difference_solution(N_grid, exact_y)

    if N_grid < 3 || floor(N_grid) ~= N_grid

        error('N_grid must be an integer greater than or equal to 3.');
```

end

```

    Delta_x = 1 / (N_grid - 1);

    x = linspace(0, 1, N_grid).';

    nUnknown = N_grid - 2;
```



```

y_left = 0;

y_right = 2;

lowerCoef = 1 / Delta_x^2 - 2 / Delta_x;

centerCoef = -2 / Delta_x^2 + 4;

upperCoef = 1 / Delta_x^2 + 2 / Delta_x;

A = zeros(nUnknown, nUnknown);

d = x(2:N_grid - 1);

for j = 1:nUnknown

    A(j, j) = centerCoef;

    if j > 1

        A(j, j - 1) = lowerCoef;

    else

        d(j) = d(j) - lowerCoef * y_left;

    end

    if j < nUnknown

        A(j, j + 1) = upperCoef;

    else

        d(j) = d(j) - upperCoef * y_right;

    end

end
end

```

```

    yInterior = gauss_pp_solve(A, d);

    y = [y_left; yInterior; y_right];

    e = y - exact_y(x);

end

function [x, y, e, theta] = shooting_solution(N_grid, method, exact_y)

    if N_grid < 2 || floor(N_grid) ~= N_grid

        error('N_grid must be an integer greater than or equal to 2.');
```

end

```

    x = linspace(0, 1, N_grid).';

    U = march_ivp(N_grid, method, [0; 0], false);

    V = march_ivp(N_grid, method, [0; 1], true);

    denominator = V(end, 1);

    if abs(denominator) < 100 * eps

        error('shooting denominator is too small.');
```

end

```

    theta = (2 - U(end, 1)) / denominator;

    y = U(:, 1) + theta * V(:, 1);

    e = y - exact_y(x);
```

```
end
```

```
function Y = march_ivp(N_grid, method, initialState, homogeneous)
```

```
    Delta_x = 1 / (N_grid - 1);
```

```
    x = linspace(0, 1, N_grid).';
```

```
    Y = zeros(N_grid, 2);
```

```
    u = initialState(:);
```

```
    Y(1, :) = u.');
```

```
    for j = 1:N_grid - 1
```

```
        switch method
```

```
            case 'ForwardEuler'
```

```
                u = u + Delta_x * ode_rhs(x(j), u, homogeneous);
```

```
            case 'CrankNicolson'
```

```
                u = crank_nicolson_step(x(j), Delta_x, u, homogeneous);
```

```
            otherwise
```

```
                error('Unknown time marching method.');
```

```
        end
```

```
        Y(j + 1, :) = u.');
```

```
    end
```

```
end
```

```
function f = ode_rhs(x, u, homogeneous)
```

```
    if homogeneous
```

```
        forcing = 0;
```

```
    else
```

```
        forcing = x;
```

```
    end
```

```
    f = [u(2); forcing - 4 * u(2) - 4 * u(1)];
```

```
end
```

```
function uNext = crank_nicolson_step(x, Delta_x, u, homogeneous)
```

```
    if homogeneous
```

```
        forcingNow = 0;
```

```
        forcingNext = 0;
```

```
    else
```

```
        forcingNow = x;
```

```
        forcingNext = x + Delta_x;
```

```
    end
```

```
    rhs1 = u(1) + 0.5 * Delta_x * u(2);
```

```
    rhs2 = u(2) - 2 * Delta_x * u(1) - 2 * Delta_x * u(2) + 0.5 * Delta_x *
```

```
(forcingNow + forcingNext);
```

```
    b11 = 1;
```

```

b12 = -0.5 * Delta_x;

b21 = 2 * Delta_x;

b22 = 1 + 2 * Delta_x;


detB = b11 * b22 - b12 * b21;


if abs(detB) < 100 * eps

    error('Crank-Nicolson step matrix is singular or nearly singular.');
```

end

```

uNext = [(b22 * rhs1 - b12 * rhs2) / detB; ...

        (-b21 * rhs1 + b11 * rhs2) / detB];

end


function x = gauss_pp_solve(A, b)

    A = double(A);

    b = double(b(:));

    n = length(b);

    if size(A, 1) ~= n || size(A, 2) ~= n

        error('A must be square and compatible with b.');
```

end

```

    Ab = [A, b];
```

```

tol = 100 * eps * (norm(A, inf) + norm(b, inf) + 1);

for k = 1:n - 1

    [pivotAbs, pivotRel] = max(abs(Ab(k:n, k)));

    pivot = pivotRel + k - 1;

    if pivotAbs < tol

        error('Matrix is singular or nearly singular.');
```

end

```

    if pivot ~= k

        temp = Ab(k, :);

        Ab(k, :) = Ab(pivot, :);

        Ab(pivot, :) = temp;

    end

    for i = k + 1:n

        factor = Ab(i, k) / Ab(k, k);

        Ab(i, k:n + 1) = Ab(i, k:n + 1) - factor * Ab(k, k:n + 1);

    end

end

if abs(Ab(n, n)) < tol

    error('Matrix is singular or nearly singular.');
```

end

```

x = zeros(n, 1);

for i = n:-1:1

    s = Ab(i, n + 1);

    if i < n

        s = s - Ab(i, i + 1:n) * x(i + 1:n);

    end

    x(i) = s / Ab(i, i);

end

end

function p = observed_order(Delta_x_values, error_values)

    p = log(error_values(1:end - 1) ./ error_values(2:end)) ./
log(Delta_x_values(1:end - 1) ./ Delta_x_values(2:end));

end

function save_current_figure(outDir, baseName, saveFigures)

    if saveFigures

        saveas(gcf, fullfile(outDir, [baseName, '.png']));

        savefig(gcf, fullfile(outDir, [baseName, '.fig']));

    end

end

end

```

舊版:Q3 AI link: <https://chatgpt.com/share/6a19f240-b760-8322-a68e-b06ec29b6e24>

Prompt: 我正在完成本課程期末專題第三題。根據課程規範，AI 只能協助撰寫或除錯對應的程式碼，不能協助撰寫報告內容、進行結果分析、解釋物理意義、翻譯或潤飾文字。

請你只協助我撰寫第三題所需的程式碼。

使用的程式語言：程式碼使用 MATLAB (但(4)模擬部分看你認為什麼軟體效果好或方便,你就告訴我要用什麼軟體,可以用 MATLAB、Wolfram Mathematica,或者是用 Python,什麼都好,你認為哪個程式碼模擬起來效果好,就用哪一個。)

1. 請只提供完成計算任務所需的程式碼。
2. 請不要撰寫任何報告段落。
3. 請不要分析或解釋數值結果。
4. 請不要提供討論、結論、物理意義或科學寫作內容。
5. 請不要翻譯或潤飾任何報告文字。
6. 可以在程式碼中加入簡潔註解，用來說明程式邏輯。
7. 程式碼需要清楚、可重現、容易修改。
8. 圖形需要包含清楚的座標軸標籤、圖例、單位與標題；若合適，請加入儲存圖檔或輸出資料的指令。

Q3 code:

```
clear; close all; clc;

format long e

outDir = 'q3_outputs';

saveFigures = true;

saveData = true;

meshList = [6 2; 12 4; 24 8; 48 16; 96 32];

loadType = 'point_center';

essentialBC = 'zero_clamp';

solverType = 'chol';
```



```
deformationScale = [];  
  
if ~exist(outDir, 'dir')  
    mkdir(outDir);  
end  
  
par.P = -8.0e3;  
  
par.L = 60.0;  
  
par.a = 10.0;  
  
par.b = 1.0;  
  
par.E = 100.0e3;  
  
par.nu = 0.3;  
  
par.G = par.E / (2 * (1 + par.nu));  
  
par.I = 2 * par.b * par.a^3 / 3;  
  
par.D = par.E / (1 - par.nu^2) * [1, par.nu, 0; par.nu, 1, 0; 0, 0, (1 - par.nu) / 2];  
  
nCase = size(meshList, 1);  
  
case_id = (1:nCase).';  
  
Nx_col = meshList(:, 1);  
  
Ny_col = meshList(:, 2);  
  
n_nodes_col = zeros(nCase, 1);  
  
n_elements_col = zeros(nCase, 1);  
  
n_dof_col = zeros(nCase, 1);  
  
hx_mm_col = zeros(nCase, 1);
```

```
hy_mm_col = zeros(nCase, 1);

h_mm_col = zeros(nCase, 1);

nnz_K_col = zeros(nCase, 1);

K_density_col = zeros(nCase, 1);

K_bandwidth_col = zeros(nCase, 1);

K_symmetry_relative_error_col = zeros(nCase, 1);

chol_flag_free_col = zeros(nCase, 1);

condest_Kfree_col = zeros(nCase, 1);

total_F2_N_col = zeros(nCase, 1);

free_end_u2_mm_col = zeros(nCase, 1);

exact_free_end_u2_mm_col = zeros(nCase, 1);

free_end_u2_abs_error_mm_col = zeros(nCase, 1);

max_nodal_error_mm_col = zeros(nCase, 1);

rms_nodal_error_mm_col = zeros(nCase, 1);

max_interior_nodal_error_mm_col = zeros(nCase, 1);

L2_displacement_error_mm_col = zeros(nCase, 1);

relative_L2_displacement_error_col = zeros(nCase, 1);

relative_L2_stress_error_col = zeros(nCase, 1);

solver_flag_col = zeros(nCase, 1);

solver_relres_col = zeros(nCase, 1);

solver_iter_col = zeros(nCase, 1);


meshes = cell(nCase, 1);

solutions = cell(nCase, 1);

forces = cell(nCase, 1);
```

```

errorData = cell(nCase, 1);

stressData = cell(nCase, 1);

for ic = 1:nCase

    Nx = meshList(ic, 1);

    Ny = meshList(ic, 2);

    mesh = createRectMesh(par.L, par.a, Nx, Ny);

    [K, F] = assembleSystem(mesh, par);

    F = applyLoad(F, mesh, par, loadType);

    [fixedDofs, fixedValues] = makeEssentialBC(mesh, par, essentialBC);

    [U, solverInfo] = solveWithEssentialBC(K, F, fixedDofs, fixedValues, solverType);

    err = computeErrorNorms(mesh, U, par);

    elemStress = computeElementStress(mesh, U, par);

    [~, u2ExactFreeEnd] = exactDisplacement(0.0, 0.0, par);

    u2FreeEnd = interpolatePointDisplacement(mesh, U, 0.0, 0.0, 2);

    [rowK, colK] = find(K);

    kff = K(setdiff((1:2*mesh.nNode).', fixedDofs), setdiff((1:2*mesh.nNode).',
fixedDofs));

    n_nodes_col(ic) = mesh.nNode;

```

```

n_elements_col(ic) = mesh.nElem;

n_dof_col(ic) = 2 * mesh.nNode;

hx_mm_col(ic) = mesh.hx;

hy_mm_col(ic) = mesh.hy;

h_mm_col(ic) = max(mesh.hx, mesh.hy);

nnz_K_col(ic) = nnz(K);

K_density_col(ic) = nnz(K) / numel(K);

K_bandwidth_col(ic) = max(abs(rowK - colK));

K_symmetry_relative_error_col(ic) = norm(K - K.', 'fro') / max(norm(K, 'fro'),
eps);

chol_flag_free_col(ic) = solverInfo.chol_flag;

condest_Kfree_col(ic) = safeCondest(Kff);

total_F2_N_col(ic) = sum(F(2:2:end));

free_end_u2_mm_col(ic) = u2FreeEnd;

exact_free_end_u2_mm_col(ic) = u2ExactFreeEnd;

free_end_u2_abs_error_mm_col(ic) = abs(u2FreeEnd - u2ExactFreeEnd);

max_nodal_error_mm_col(ic) = err.maxNodalDispError;

rms_nodal_error_mm_col(ic) = err.rmsNodalDispError;

max_interior_nodal_error_mm_col(ic) = err.maxInteriorNodalDispError;

L2_displacement_error_mm_col(ic) = err.L2DispError;

relative_L2_displacement_error_col(ic) = err.relL2DispError;

relative_L2_stress_error_col(ic) = err.relL2StressError;

solver_flag_col(ic) = solverInfo.flag;

solver_relres_col(ic) = solverInfo.relres;

solver_iter_col(ic) = solverInfo.iter;

```

```

    meshes{ic} = mesh;

    solutions{ic} = U;

    forces{ic} = F;

    errorData{ic} = err;

    stressData{ic} = elemStress;

end

reference_rms_error_mm_col = zeros(nCase, 1);

reference_relative_rms_error_col = zeros(nCase, 1);

meshRef = meshes{end};

URef = solutions{end};

for ic = 1:nCase

    mesh = meshes{ic};

    U = solutions{ic};

    URefAtNodes = interpolateSolutionAtPoints(meshRef, URef, mesh.coords);

    UAtNodes = [U(1:2:end), U(2:2:end)];

    diffU = UAtNodes - URefAtNodes;

    reference_rms_error_mm_col(ic) = sqrt(mean(sum(diffU.^2, 2)));

    reference_relative_rms_error_col(ic) = reference_rms_error_mm_col(ic) /

max(sqrt(mean(sum(URefAtNodes.^2, 2))), eps);

```

end

```
rate_exact_relative_L2 = convergenceRate(h_mm_col,  
relative_L2_displacement_error_col);  
  
rate_reference_relative_rms = convergenceRate(h_mm_col,  
reference_relative_rms_error_col);  
  
summary_table = table(case_id, Nx_col, Ny_col, n_nodes_col, n_elements_col,  
n_dof_col, ...  
    hx_mm_col, hy_mm_col, h_mm_col, nnz_K_col, K_density_col, K_bandwidth_col, ...  
    K_symmetry_relative_error_col, chol_flag_free_col, condest_Kfree_col,  
total_F2_N_col, ...  
    free_end_u2_mm_col, exact_free_end_u2_mm_col, free_end_u2_abs_error_mm_col, ...  
    max_nodal_error_mm_col, rms_nodal_error_mm_col,  
max_interior_nodal_error_mm_col, ...  
    L2_displacement_error_mm_col, relative_L2_displacement_error_col,  
relative_L2_stress_error_col, ...  
    reference_rms_error_mm_col, reference_relative_rms_error_col, ...  
    rate_exact_relative_L2, rate_reference_relative_rms, solver_flag_col,  
solver_relres_col, solver_iter_col);  
  
disp(summary_table)  
  
if saveData  
    writetable(summary_table, fullfile(outDir, 'q3_summary.csv'));
```

```
end
```

```
mesh = meshes{end};
```

```
U = solutions{end};
```

```
elemStress = stressData{end};
```

```
[u1ExactNode, u2ExactNode] = exactDisplacement(mesh.coords(:, 1), mesh.coords(:, 2),  
par);
```

```
Uxy = [U(1:2:end), U(2:2:end)];
```

```
nodalErr = Uxy - [u1ExactNode, u2ExactNode];
```

```
nodal_table = table((1:mesh.nNode).', mesh.coords(:, 1), mesh.coords(:, 2), ...
```

```
    Uxy(:, 1), Uxy(:, 2), u1ExactNode, u2ExactNode, ...
```

```
    nodalErr(:, 1), nodalErr(:, 2), sqrt(sum(nodalErr.^2, 2)), ...
```

```
    'VariableNames', {'node_id', 'x1_mm', 'x2_mm', ...
```

```
    'u1_FE_mm', 'u2_FE_mm', 'u1_exact_mm', 'u2_exact_mm', ...
```

```
    'u1_error_mm', 'u2_error_mm', 'displacement_error_norm_mm'}));
```

```
element_table = table((1:mesh.nElem).', elemStress.x1Center, elemStress.x2Center, ...
```

```
    elemStress.sigma1, elemStress.sigma2, elemStress.tau12, elemStress.vonMises, ...
```

```
    elemStress.sigma1Exact, elemStress.sigma2Exact, elemStress.tau12Exact,
```

```
elemStress.vonMisesExact, ...
```

```
    'VariableNames', {'element_id', 'x1_center_mm', 'x2_center_mm', ...
```

```
    'sigma1_N_per_mm2', 'sigma2_N_per_mm2', 'tau12_N_per_mm2',
```

```
    'von_mises_N_per_mm2', ...
```

```

        'sigma1_exact_N_per_mm2', 'sigma2_exact_N_per_mm2', 'tau12_exact_N_per_mm2',
        'von_mises_exact_N_per_mm2'}));

if saveData

    writetable(nodal_table, fullfile(outDir, 'q3_nodal_solution_finetest_mesh.csv'));

    writetable(element_table, fullfile(outDir, 'q3_element_stress_finetest_mesh.csv'));

    save(fullfile(outDir, 'q3_workspace_results.mat'), ...

        'par', 'meshList', 'loadType', 'essentialBC', 'solverType', ...

        'summary_table', 'nodal_table', 'element_table', ...

        'meshes', 'solutions', 'forces', 'errorData', 'stressData');

end

figure('color', 'w');

loglog(h_mm_col, relative_L2_displacement_error_col, 'o-', 'Linewidth', 1.5,

'MarkerSize', 6); hold on;

loglog(h_mm_col, reference_relative_rms_error_col, 's-', 'Linewidth', 1.5,

'MarkerSize', 6);

set(gca, 'xDir', 'reverse');

grid on;

xlabel('h [mm]');

ylabel('Relative displacement error [-]');

title('Q3 spatial convergence');

legend('Relative L2 error vs exact field', 'Relative RMS error vs finest mesh',

'Location', 'best');

saveCurrentFigure(outDir, 'q3_spatial_convergence', saveFigures);

```



```

figure('Color', 'w');

plot(h_mm_col, free_end_u2_mm_col, 'o-', 'Linewidth', 1.5, 'MarkerSize', 6); hold on;

plot(h_mm_col, exact_free_end_u2_mm_col, 'k--', 'Linewidth', 1.5);

set(gca, 'xDir', 'reverse');

grid on;

xlabel('h [mm]');

ylabel('u_2(0,0) [mm]');

title('Q3 free-end displacement');

legend('Finite element', 'Exact field value', 'Location', 'best');

saveCurrentFigure(outDir, 'q3_free_end_displacement', saveFigures);


plotDeformedMesh(mesh, U, par, deformationScale, outDir, saveFigures);

plotNodalContour(mesh, U(1:2:end), 'u_1 [mm]', 'Q3 displacement u_1', 'q3_contour_u1',
outDir, saveFigures);

plotNodalContour(mesh, U(2:2:end), 'u_2 [mm]', 'Q3 displacement u_2', 'q3_contour_u2',
outDir, saveFigures);

plotElementContour(mesh, elemStress.sigma1, '\sigma_1 [N/mm^2]', 'Q3 element stress
\sigma_1', 'q3_contour_sigma1', outDir, saveFigures);

plotElementContour(mesh, elemStress.tau12, '\tau_{12} [N/mm^2]', 'Q3 element stress
\tau_{12}', 'q3_contour_tau12', outDir, saveFigures);


function mesh = createRectMesh(L, a, Nx, Ny)

    x = linspace(0, L, Nx + 1);

    y = linspace(-a, a, Ny + 1);

```

```

[x1, x2] = ndgrid(x, y);

coords = [x1(:), x2(:)];

nodeId = @(i, j) i + 1 + j * (Nx + 1);

elem = zeros(Nx * Ny, 4);

e = 0;

for j = 0:Ny - 1

    for i = 0:Nx - 1

        e = e + 1;

        elem(e, :) = [nodeId(i, j), nodeId(i + 1, j), nodeId(i + 1, j + 1),
nodeId(i, j + 1)];

    end

end

mesh.coords = coords;

mesh.elem = elem;

mesh.Nx = Nx;

mesh.Ny = Ny;

mesh.nNode = size(coords, 1);

mesh.nElem = size(elem, 1);

mesh.nodeId = nodeId;

mesh.xgrid = x;

mesh.ygrid = y;

```

```
mesh.hx = L / Nx;  
  
mesh.hy = 2 * a / Ny;  
  
mesh.L = L;  
  
mesh.a = a;
```

```
end
```

```
function [K, F] = assembleSystem(mesh, par)
```

```
    ndof = 2 * mesh.nNode;  
  
    F = zeros(ndof, 1);  
  
  
  
    Iidx = zeros(64 * mesh.nElem, 1);  
    Jidx = zeros(64 * mesh.nElem, 1);  
    vals = zeros(64 * mesh.nElem, 1);  
  
    counter = 0;  
  
  
    gp = [-1 / sqrt(3), -1 / sqrt(3);  
          1 / sqrt(3), -1 / sqrt(3);  
          1 / sqrt(3), 1 / sqrt(3);  
          -1 / sqrt(3), 1 / sqrt(3)];  
  
    gw = [1; 1; 1; 1];  
  
  
    for e = 1:mesh.nElem  
  
        nodes = mesh.elem(e, :);  
  
        xe = mesh.coords(nodes, :);  
  
        ke = zeros(8, 8);
```

```

for q = 1:4

    xi = gp(q, 1);

    eta = gp(q, 2);

    [~, dN_dxi] = shapeQ4(xi, eta);

    Jmat = Xe.' * dN_dxi;

    detJ = det(Jmat);

    if detJ <= 0

        error('Element %d has non-positive Jacobian determinant.', e);

    end

    dN_dx = dN_dxi / Jmat;

    B = buildB(dN_dx);

    Ke = Ke + par.b * (B.' * par.D * B) * detJ * gw(q);

end

edofs = elementDofs(nodes);

for ii = 1:8

    for jj = 1:8

        counter = counter + 1;

        Iidx(counter) = edofs(ii);

        Jidx(counter) = edofs(jj);

```

```

        vals(counter) = ke(ii, jj);

    end

end

end

K = sparse(Iidx, Jidx, Vals, ndof, ndof);

K = 0.5 * (K + K. ');

end

function F = applyLoad(F, mesh, par, loadType)

    switch lower(loadType)

        case 'point_center'

            F = applyPointLoad(F, mesh, 0.0, 0.0, 2, par.P);

        case 'parabolic_traction'

            F = applyParabolicEndTraction(F, mesh, par);

        otherwise

            error('Unknown loadType. ');

        end

    end

end

function F = applyPointLoad(F, mesh, xLoad, yLoad, component, value)

    tol = 1.0e-10 * max([1.0, mesh.L, mesh.a]);

    edgeNodes = find(abs(mesh.coords(:, 1) - xLoad) <= tol);

```

```

if isempty(edgeNodes)

    error('No edge node found at the requested xLoad.');
```

end

```

[ys, order] = sort(mesh.coords(edgeNodes, 2));

edgeNodes = edgeNodes(order);

[minDist, exactIdx] = min(abs(ys - yLoad));

if minDist <= tol

    targetNodes = edgeNodes(exactIdx);

    weights = 1;

else

    k = find(ys < yLoad, 1, 'last');

    if isempty(k) || k >= numel(ys)

        error('Point load y-coordinate is outside the loaded edge.');
```

end

```

    y1 = ys(k);

    y2 = ys(k + 1);

    targetNodes = [edgeNodes(k); edgeNodes(k + 1)];

    weights = [(y2 - yLoad) / (y2 - y1); (yLoad - y1) / (y2 - y1)];

end
```

```

for m = 1:numel(targetNodes)

    dof = 2 * targetNodes(m) - 2 + component;

    F(dof) = F(dof) + weights(m) * value;

end

end

function F = applyParabolicEndTraction(F, mesh, par)

    gp = [-1 / sqrt(3); 1 / sqrt(3)];

    gw = [1; 1];

    for j = 0:mesh.Ny - 1

        nBot = mesh.nodeId(0, j);

        nTop = mesh.nodeId(0, j + 1);

        yBot = mesh.coords(nBot, 2);

        yTop = mesh.coords(nTop, 2);

        jacLine = (yTop - yBot) / 2;

        for q = 1:2

            eta = gp(q);

            Nedge = [(1 - eta) / 2; (1 + eta) / 2];

            yq = Nedge(1) * yBot + Nedge(2) * yTop;

            traction2 = par.P / (2 * par.I) * (par.a^2 - yq^2);

            F(2 * nBot) = F(2 * nBot) + par.b * Nedge(1) * traction2 * jacLine *

```

```

gw(q);

    F(2 * nTop) = F(2 * nTop) + par.b * Nedge(2) * traction2 * jacLine *

gw(q);

    end

end

end

function [fixedDofs, fixedValues] = makeEssentialBC(mesh, par, essentialBC)

    tol = 1.0e-10 * max([1.0, mesh.L, mesh.a]);

    fixedNodes = find(abs(mesh.coords(:, 1) - par.L) <= tol);

    fixedDofs = zeros(2 * numel(fixedNodes), 1);

    fixedValues = zeros(2 * numel(fixedNodes), 1);

    for k = 1:numel(fixedNodes)

        node = fixedNodes(k);

        fixedDofs(2 * k - 1) = 2 * node - 1;

        fixedDofs(2 * k) = 2 * node;

    end

    switch lower(essentialBC)

        case 'zero_clamp'

            fixedValues(:) = 0;

        case 'exact_displacement'

            [u1, u2] = exactDisplacement(mesh.coords(fixedNodes, 1),

```



```
mesh.coords(fixedNodes, 2), par);
```

```
    for k = 1:numel(fixedNodes)
```

```
        fixedValues(2 * k - 1) = u1(k);
```

```
        fixedValues(2 * k) = u2(k);
```

```
    end
```

```
    otherwise
```

```
        error('Unknown essentialBC.');
```

```
    end
```

```
end
```

```
function [U, solverInfo] = solveWithEssentialBC(K, F, fixedDofs, fixedValues,  
solverType)
```

```
    allDofs = (1:numel(F)).';
```

```
    fixedDofs = fixedDofs(:);
```

```
    fixedValues = fixedValues(:);
```

```
    freeDofs = setdiff(allDofs, fixedDofs);
```

```
    U = zeros(numel(F), 1);
```

```
    U(fixedDofs) = fixedValues;
```

```
    rhs = F(freeDofs) - K(freeDofs, fixedDofs) * fixedValues;
```

```
    Kff = K(freeDofs, freeDofs);
```

```

    [Ufree, solverInfo] = solveLinearSystem(Kff, rhs, solverType);

    U(freeDofs) = Ufree;

end

```

```

function [x, solverInfo] = solveLinearSystem(A, b, solverType)

```

```

    solverInfo.method = solverType;

```

```

    solverInfo.flag = NaN;

```

```

    solverInfo.relres = NaN;

```

```

    solverInfo.iter = NaN;

```

```

    solverInfo.chol_flag = NaN;

```

```

    switch lower(solverType)

```

```

        case 'chol'

```

```

            p = symamd(A);

```

```

            Ap = A(p, p);

```

```

            bp = b(p);

```

```

            [R, cholFlag] = chol(Ap, 'upper');

```

```

            solverInfo.chol_flag = cholFlag;

```

```

            if cholFlag == 0

```

```

                xp = R \ (R.' \ bp);

```

```

                x = zeros(size(b));

```

```

                x(p) = xp;

```

```

                solverInfo.flag = 0;

```

```

        solverInfo.relres = norm(A * x - b) / max(norm(b), eps);

        solverInfo.iter = 1;

    else

        [x, solverInfoPCG] = solveLinearSystem(A, b, 'pcg');

        solverInfo.flag = solverInfoPCG.flag;

        solverInfo.relres = solverInfoPCG.relres;

        solverInfo.iter = solverInfoPCG.iter;

    end

case 'pcg'

    tol = 1.0e-10;

    maxit = max(200, min(size(A, 1), 5000));

    try

        opts.type = 'ict';

        opts.droptol = 1.0e-3;

        opts.diagcomp = 1.0e-8;

        L = ichol(A, opts);

        [x, flag, relres, iter] = pcg(A, b, tol, maxit, L, L.');
```

catch

```

        [x, flag, relres, iter] = pcg(A, b, tol, maxit);

    end

    solverInfo.flag = flag;

    solverInfo.relres = relres;

```

```

        solverInfo.iter = iter;

        solverInfo.chol_flag = NaN;

    case 'backslash'

        x = A \ b;

        solverInfo.flag = 0;

        solverInfo.relres = norm(A * x - b) / max(norm(b), eps);

        solverInfo.iter = 1;

        solverInfo.chol_flag = NaN;

    otherwise

        error('Unknown solverType.');
```

end

end

```

function err = computeErrorNorms(mesh, U, par)

    coords = mesh.coords;

    Uxy = [U(1:2:end), U(2:2:end)];

    [u1Exact, u2Exact] = exactDisplacement(coords(:, 1), coords(:, 2), par);

    nodalErr = Uxy - [u1Exact, u2Exact];

    nodalErrNorm = sqrt(sum(nodalErr.^2, 2));

    fixedMask = abs(coords(:, 1) - par.L) <= 1.0e-10 * max([1.0, par.L, par.a]);

    interiorMask = ~fixedMask;
```

```

gp = [-1 / sqrt(3), -1 / sqrt(3);
      1 / sqrt(3), -1 / sqrt(3);
      1 / sqrt(3), 1 / sqrt(3);
      -1 / sqrt(3), 1 / sqrt(3)];

gw = [1; 1; 1; 1];

intErr2 = 0;

intExact2 = 0;

intStressErr2 = 0;

intStressExact2 = 0;

for e = 1:mesh.nElem

    nodes = mesh.elem(e, :);

    Xe = coords(nodes, :);

    edofs = elementDofs(nodes);

    Ue = U(edofs);

    for q = 1:4

        xi = gp(q, 1);

        eta = gp(q, 2);

        [N, dN_dxi] = shapeQ4(xi, eta);

        Jmat = Xe.' * dN_dxi;

        detJ = det(Jmat);

```

```

    dN_dx = dN_dxi / Jmat;

    Nmat = buildNmat(N);

    B = buildB(dN_dx);

    xq = N.' * Xe(:, 1);
    yq = N.' * Xe(:, 2);

    uh = Nmat * Ue;

    strainh = B * Ue;

    stressh = par.D * strainh;

    [u1e, u2e] = exactDisplacement(xq, yq, par);

    [s1e, s2e, t12e] = exactStress(xq, yq, par);

    ue = [u1e; u2e];

    stresse = [s1e; s2e; t12e];

    intErr2 = intErr2 + sum((uh - ue).^2) * detJ * gw(q);

    intExact2 = intExact2 + sum(ue.^2) * detJ * gw(q);

    intStressErr2 = intStressErr2 + sum((stressh - stresse).^2) * detJ *
gw(q);

    intStressExact2 = intStressExact2 + sum(stresse.^2) * detJ * gw(q);

end

end

```

```

err.maxNodalDispError = max(nodalErrNorm);

err.rmsNodalDispError = sqrt(mean(nodalErrNorm.^2));

err.maxInteriorNodalDispError = max(nodalErrNorm(interiorMask));

err.L2DispError = sqrt(intErr2);

err.relL2DispError = sqrt(intErr2 / max(intExact2, eps));

err.relL2StressError = sqrt(intStressErr2 / max(intStressExact2, eps));

end

function elemStress = computeElementStress(mesh, U, par)

    elemStress.x1Center = zeros(mesh.nElem, 1);

    elemStress.x2Center = zeros(mesh.nElem, 1);

    elemStress.sigma1 = zeros(mesh.nElem, 1);

    elemStress.sigma2 = zeros(mesh.nElem, 1);

    elemStress.tau12 = zeros(mesh.nElem, 1);

    elemStress.vonMises = zeros(mesh.nElem, 1);

    elemStress.sigma1Exact = zeros(mesh.nElem, 1);

    elemStress.sigma2Exact = zeros(mesh.nElem, 1);

    elemStress.tau12Exact = zeros(mesh.nElem, 1);

    elemStress.vonMisesExact = zeros(mesh.nElem, 1);

    for e = 1:mesh.nElem

        nodes = mesh.elem(e, :);

        xe = mesh.coords(nodes, :);

        ue = U(elementDofs(nodes));

```

```

[N, dN_dxi] = shapeQ4(0.0, 0.0);

Jmat = xe.' * dN_dxi;

dN_dx = dN_dxi / Jmat;

B = buildB(dN_dx);

xq = N.' * xe(:, 1);

yq = N.' * xe(:, 2);

stress = par.D * B * Ue;

[s1e, s2e, t12e] = exactStress(xq, yq, par);

elemStress.x1Center(e) = xq;

elemStress.x2Center(e) = yq;

elemStress.sigma1(e) = stress(1);

elemStress.sigma2(e) = stress(2);

elemStress.tau12(e) = stress(3);

elemStress.vonMises(e) = planeStressVonMises(stress(1), stress(2), stress(3));

elemStress.sigma1Exact(e) = s1e;

elemStress.sigma2Exact(e) = s2e;

elemStress.tau12Exact(e) = t12e;

elemStress.vonMisesExact(e) = planeStressVonMises(s1e, s2e, t12e);

end

end

```



```

function [N, dN_dxi] = shapeQ4(xi, eta)

    N = 0.25 * [(1 - xi) * (1 - eta);

                (1 + xi) * (1 - eta);

                (1 + xi) * (1 + eta);

                (1 - xi) * (1 + eta)];

    dN_dxi = 0.25 * [-(1 - eta), -(1 - xi);

                     (1 - eta), -(1 + xi);

                     (1 + eta), (1 + xi);

                     -(1 + eta), (1 - xi)];

end

```

```

function B = buildB(dN_dx)

    B = zeros(3, 8);

    for i = 1:4

        c1 = 2 * i - 1;

        c2 = 2 * i;

        B(1, c1) = dN_dx(i, 1);

        B(2, c2) = dN_dx(i, 2);

        B(3, c1) = dN_dx(i, 2);

        B(3, c2) = dN_dx(i, 1);

    end

end

```

```
function Nmat = buildNmat(N)
```

```
    Nmat = zeros(2, 8);
```

```
    for i = 1:4
```

```
        Nmat(1, 2 * i - 1) = N(i);
```

```
        Nmat(2, 2 * i) = N(i);
```

```
    end
```

```
end
```

```
function dofs = elementDofs(nodes)
```

```
    nodes = nodes(:);
```

```
    dofs = zeros(8, 1);
```

```
    dofs(1:2:end) = 2 * nodes - 1;
```

```
    dofs(2:2:end) = 2 * nodes;
```

```
end
```

```
function [u1, u2] = exactDisplacement(x1, x2, par)
```

```
    P = par.P;
```

```
    L = par.L;
```

```
    a = par.a;
```

```
    E = par.E;
```

```
    I = par.I;
```

```
    nu = par.nu;
```

```
    G = par.G;
```

```

u1 = -P ./ (2 * E * I) .* (x1.^2 - L^2) .* x2 ...

      + nu * P ./ (6 * E * I) .* x2.^3 ...

      - P ./ (2 * G * I) .* (a^2 .* x2 - x2.^3 ./ 3);

u2 = P ./ (6 * E * I) .* (x1.^3 - 3 * L^2 .* x1 + 2 * L^3) ...

      + nu * P ./ (2 * E * I) .* (L - x1) .* x2.^2 ...

      + P * a^2 ./ (2 * G * I) .* (L - x1);

end

function [sigma1, sigma2, tau12] = exactStress(x1, x2, par)

    P = par.P;

    a = par.a;

    I = par.I;

    sigma1 = -P .* x1 .* x2 ./ I;

    sigma2 = zeros(size(x1));

    tau12 = -P ./ (2 * I) .* (a^2 - x2.^2);

end

function vm = planeStressvonMises(sigma1, sigma2, tau12)

    vm = sqrt(sigma1.^2 - sigma1 .* sigma2 + sigma2.^2 + 3 * tau12.^2);

end

function val = interpolatePointDisplacement(mesh, U, xq, yq, component)

```

```

    Uq = interpolateSolutionAtPoints(mesh, U, [xq, yq]);

    val = Uq(1, component);

end

function Uq = interpolateSolutionAtPoints(mesh, U, queryCoords)

    nq = size(queryCoords, 1);

    Uq = zeros(nq, 2);

    for q = 1:nq

        xq = queryCoords(q, 1);

        yq = queryCoords(q, 2);

        i = floor((xq - mesh.xgrid(1)) / mesh.hx);

        j = floor((yq - mesh.ygrid(1)) / mesh.hy);

        i = max(0, min(mesh.Nx - 1, i));

        j = max(0, min(mesh.Ny - 1, j));

        xLeft = mesh.xgrid(i + 1);

        yBottom = mesh.ygrid(j + 1);

        xi = 2 * (xq - xLeft) / mesh.hx - 1;

        eta = 2 * (yq - yBottom) / mesh.hy - 1;

        xi = max(-1, min(1, xi));

```

```

    eta = max(-1, min(1, eta));

    [N, ~] = shapeQ4(xi, eta);

    nodes = [mesh.nodeId(i, j), mesh.nodeId(i + 1, j), ...
             mesh.nodeId(i + 1, j + 1), mesh.nodeId(i, j + 1)];

    ux = U(2 * nodes - 1);

    uy = U(2 * nodes);

    Uq(q, 1) = N.' * ux(:);

    Uq(q, 2) = N.' * uy(:);

end

end

function rate = convergenceRate(h, err)

    n = numel(h);

    rate = NaN(n, 1);

    for i = 2:n

        if err(i - 1) > 0 && err(i) > 0 && h(i - 1) > h(i)

            rate(i) = log(err(i - 1) / err(i)) / log(h(i - 1) / h(i));

        end

    end

end

end

```

```

function c = safeCondest(A)

    try

        c = condest(A);

    catch

        c = NaN;

    end

end

function plotDeformedMesh(mesh, U, par, scaleFactor, outDir, saveFigures)

    Uxy = [U(1:2:end), U(2:2:end)];

    maxDisp = max(sqrt(sum(Uxy.^2, 2)));

    if isempty(scaleFactor)

        scaleFactor = 0.2 * par.L / max(maxDisp, eps);

    end

    defCoords = mesh.coords + scaleFactor * Uxy;

    figure('Color', 'w');

    patch('Faces', mesh.elem, 'Vertices', mesh.coords, ...

        'FaceColor', 'none', 'EdgeColor', [0.65 0.65 0.65], 'Linewidth', 0.6);

    hold on;

    patch('Faces', mesh.elem, 'Vertices', defCoords, ...

        'FaceColor', 'none', 'EdgeColor', [0.00 0.20 0.80], 'Linewidth', 0.8);

```

```

    grid on;

    axis equal tight;

    xlabel('x_1 [mm]');

    ylabel('x_2 [mm]');

    title(sprintf('Q3 undeformed and deformed mesh, scale = %.4g', scaleFactor));

    legend('Undeformed mesh', 'Deformed mesh', 'Location', 'best');

    saveCurrentFigure(outDir, 'q3_deformed_mesh', saveFigures);

end

function plotNodalContour(mesh, nodalValue, colorbarLabel, figTitle, fileName, outDir,
saveFigures)

    figure('Color', 'w');

    patch('Faces', mesh.elem, 'Vertices', mesh.coords, ...

        'FaceVertexCData', nodalValue(:), 'FaceColor', 'interp', ...

        'EdgeColor', [0.70 0.70 0.70], 'Linewidth', 0.3);

    axis equal tight;

    grid on;

    xlabel('x_1 [mm]');

    ylabel('x_2 [mm]');

    title(figTitle);

    cb = colorbar;

    ylabel(cb, colorbarLabel);

    saveCurrentFigure(outDir, fileName, saveFigures);

end

```

```

function plotElementContour(mesh, elemValue, colorbarLabel, figTitle, fileName,
outDir, saveFigures)

    figure('color', 'w');

    patch('Faces', mesh.elem, 'Vertices', mesh.coords, ...

        'FaceColor', 'flat', 'CData', elemValue(:), ...

        'EdgeColor', 'none');

    axis equal tight;

    grid on;

    xlabel('x_1 [mm]');

    ylabel('x_2 [mm]');

    title(figTitle);

    cb = colorbar;

    ylabel(cb, colorbarLabel);

    saveCurrentFigure(outDir, fileName, saveFigures);

end

function saveCurrentFigure(outDir, fileName, saveFigures)

    if saveFigures

        try

            exportgraphics(gcf, fullfile(outDir, [fileName, '.png']), 'Resolution',
300);

        catch

            saveas(gcf, fullfile(outDir, [fileName, '.png']));

        end

        savefig(gcf, fullfile(outDir, [fileName, '.fig']));

```



```
end
```

```
end
```

新版:Q3 AI link: <https://chatgpt.com/share/6a19f240-b760-8322-a68e-b06ec29b6e24>

Prompt: 第三題 bonus 的載重設定有改,幫我把第三題修改成新版的程式碼

Q3 code:

```
clear; close all; clc;

format long e

outDir = 'q3_bonus_new_load_outputs';

saveFigures = true;

saveData = true;

meshList = [6 2; 12 4; 24 8; 48 16; 96 32];

solverType = 'chol';

essentialBC = 'zero_clamp';

deformationScale = [];

if ~exist(outDir, 'dir')
    mkdir(outDir);
end

par.P = -8.0e3;

par.L = 60.0;

par.a = 10.0;

par.b = 1.0;
```

```

par.E = 100.0e3;

par.nu = 0.3;

par.G = par.E / (2 * (1 + par.nu));

par.I = 2 * par.b * par.a^3 / 3;

par.D = par.E / (1 - par.nu^2) * [1, par.nu, 0; par.nu, 1, 0; 0, 0, (1 - par.nu) / 2];

nCase = size(meshList, 1);

case_id = (1:nCase).';

Nx_col = meshList(:, 1);

Ny_col = meshList(:, 2);

n_nodes_col = zeros(nCase, 1);

n_elements_col = zeros(nCase, 1);

n_dof_col = zeros(nCase, 1);

hx_mm_col = zeros(nCase, 1);

hy_mm_col = zeros(nCase, 1);

h_mm_col = zeros(nCase, 1);

nnz_K_col = zeros(nCase, 1);

K_density_col = zeros(nCase, 1);

K_bandwidth_col = zeros(nCase, 1);

K_symmetry_relative_error_col = zeros(nCase, 1);

chol_flag_free_col = zeros(nCase, 1);

condest_Kfree_col = zeros(nCase, 1);

total_F1_N_col = zeros(nCase, 1);

total_F2_N_col = zeros(nCase, 1);

```

```

free_end_u2_mm_col = zeros(nCase, 1);

exact_free_end_u2_mm_col = zeros(nCase, 1);

free_end_u2_abs_error_mm_col = zeros(nCase, 1);

max_nodal_error_mm_col = zeros(nCase, 1);

rms_nodal_error_mm_col = zeros(nCase, 1);

max_interior_nodal_error_mm_col = zeros(nCase, 1);

L2_displacement_error_mm_col = zeros(nCase, 1);

relative_L2_displacement_error_col = zeros(nCase, 1);

relative_L2_stress_error_col = zeros(nCase, 1);

solver_flag_col = zeros(nCase, 1);

solver_relres_col = zeros(nCase, 1);

solver_iter_col = zeros(nCase, 1);


meshes = cell(nCase, 1);

solutions = cell(nCase, 1);

forces = cell(nCase, 1);

errorData = cell(nCase, 1);

stressData = cell(nCase, 1);


for ic = 1:nCase

    Nx = meshList(ic, 1);

    Ny = meshList(ic, 2);


    mesh = createRectMesh(par.L, par.a, Nx, Ny);

    [K, F] = assembleSystem(mesh, par);

```

```

F = applyUniformFreeEndLoad(F, mesh, par);

[fixedDofs, fixedValues] = makeEssentialBC(mesh, par, essentialBC);

[U, solverInfo] = solveWithEssentialBC(K, F, fixedDofs, fixedValues, solverType);

err = computeErrorNorms(mesh, U, par);

elemStress = computeElementStress(mesh, U, par);

[~, u2ExactFreeEnd] = exactDisplacement(0.0, 0.0, par);

u2FreeEnd = interpolatePointDisplacement(mesh, U, 0.0, 0.0, 2);

[rowK, colK] = find(K);

freeDofs = setdiff((1:2*mesh.nNode).', fixedDofs);

Kff = K(freeDofs, freeDofs);

n_nodes_col(ic) = mesh.nNode;

n_elements_col(ic) = mesh.nElem;

n_dof_col(ic) = 2 * mesh.nNode;

hx_mm_col(ic) = mesh.hx;

hy_mm_col(ic) = mesh.hy;

h_mm_col(ic) = max(mesh.hx, mesh.hy);

nnz_K_col(ic) = nnz(K);

K_density_col(ic) = nnz(K) / numel(K);

K_bandwidth_col(ic) = max(abs(rowK - colK));

K_symmetry_relative_error_col(ic) = norm(K - K.', 'fro') / max(norm(K, 'fro'),

```

```

eps);

chol_flag_free_col(ic) = solverInfo.chol_flag;

condest_Kfree_col(ic) = safeCondest(Kff);

total_F1_N_col(ic) = sum(F(1:2:end));

total_F2_N_col(ic) = sum(F(2:2:end));

free_end_u2_mm_col(ic) = u2FreeEnd;

exact_free_end_u2_mm_col(ic) = u2ExactFreeEnd;

free_end_u2_abs_error_mm_col(ic) = abs(u2FreeEnd - u2ExactFreeEnd);

max_nodal_error_mm_col(ic) = err.maxNodalDispError;

rms_nodal_error_mm_col(ic) = err.rmsNodalDispError;

max_interior_nodal_error_mm_col(ic) = err.maxInteriorNodalDispError;

L2_displacement_error_mm_col(ic) = err.L2DispError;

relative_L2_displacement_error_col(ic) = err.relL2DispError;

relative_L2_stress_error_col(ic) = err.relL2StressError;

solver_flag_col(ic) = solverInfo.flag;

solver_relres_col(ic) = solverInfo.relres;

solver_iter_col(ic) = solverInfo.iter;


meshes{ic} = mesh;

solutions{ic} = U;

forces{ic} = F;

errorData{ic} = err;

stressData{ic} = elemStress;

end

```

```

reference_rms_error_mm_col = zeros(nCase, 1);

reference_relative_rms_error_col = zeros(nCase, 1);


meshRef = meshes{end};

URef = solutions{end};


for ic = 1:nCase

    mesh = meshes{ic};

    U = solutions{ic};


    URefAtNodes = interpolateSolutionAtPoints(meshRef, URef, mesh.coords);

    UAtNodes = [U(1:2:end), U(2:2:end)];

    diffU = UAtNodes - URefAtNodes;


    reference_rms_error_mm_col(ic) = sqrt(mean(sum(diffU.^2, 2)));

    reference_relative_rms_error_col(ic) = reference_rms_error_mm_col(ic) /
max(sqrt(mean(sum(URefAtNodes.^2, 2))), eps);

end


rate_exact_relative_L2 = convergenceRate(h_mm_col,
relative_L2_displacement_error_col);

rate_reference_relative_rms = convergenceRate(h_mm_col,
reference_relative_rms_error_col);


summary_table = table(case_id, Nx_col, Ny_col, n_nodes_col, n_elements_col,

```

```

n_dof_col, ...

    hx_mm_col, hy_mm_col, h_mm_col, nnz_K_col, K_density_col, K_bandwidth_col, ...

    K_symmetry_relative_error_col, chol_flag_free_col, condest_kfree_col, ...

    total_F1_N_col, total_F2_N_col, ...

    free_end_u2_mm_col, exact_free_end_u2_mm_col, free_end_u2_abs_error_mm_col, ...

    max_nodal_error_mm_col, rms_nodal_error_mm_col,

max_interior_nodal_error_mm_col, ...

    L2_displacement_error_mm_col, relative_L2_displacement_error_col,

relative_L2_stress_error_col, ...

    reference_rms_error_mm_col, reference_relative_rms_error_col, ...

    rate_exact_relative_L2, rate_reference_relative_rms, solver_flag_col,

solver_relres_col, solver_iter_col);

disp(summary_table)

if saveData

    writetable(summary_table, fullfile(outDir, 'q3_bonus_new_load_summary.csv'));

end

mesh = meshes{end};

U = solutions{end};

F = forces{end};

elemStress = stressData{end};

[u1ExactNode, u2ExactNode] = exactDisplacement(mesh.coords(:, 1), mesh.coords(:, 2),

```

```

par);

Uxy = [U(1:2:end), U(2:2:end)];

nodalErr = Uxy - [u1ExactNode, u2ExactNode];

nodal_table = table((1:mesh.nNode).', mesh.coords(:, 1), mesh.coords(:, 2), ...

    F(1:2:end), F(2:2:end), ...

    Uxy(:, 1), Uxy(:, 2), u1ExactNode, u2ExactNode, ...

    nodalErr(:, 1), nodalErr(:, 2), sqrt(sum(nodalErr.^2, 2)), ...

    'VariableNames', {'node_id', 'x1_mm', 'x2_mm', ...

    'F1_N', 'F2_N', ...

    'u1_FE_mm', 'u2_FE_mm', 'u1_exact_mm', 'u2_exact_mm', ...

    'u1_error_mm', 'u2_error_mm', 'displacement_error_norm_mm'}));

element_table = table((1:mesh.nElem).', elemStress.x1Center, elemStress.x2Center, ...

    elemStress.sigma1, elemStress.sigma2, elemStress.tau12, elemStress.vonMises, ...

    elemStress.sigma1Exact, elemStress.sigma2Exact, elemStress.tau12Exact,

    elemStress.vonMisesExact, ...

    'VariableNames', {'element_id', 'x1_center_mm', 'x2_center_mm', ...

    'sigma1_N_per_mm2', 'sigma2_N_per_mm2', 'tau12_N_per_mm2',

    'von_mises_N_per_mm2', ...

    'sigma1_exact_N_per_mm2', 'sigma2_exact_N_per_mm2', 'tau12_exact_N_per_mm2',

    'von_mises_exact_N_per_mm2'}));

if saveData

    writetable(nodal_table, fullfile(outDir,

```



```

'q3_bonus_new_load_nodal_solution_finetest_mesh.csv')));

    writetable(element_table, fullfile(outDir,
'q3_bonus_new_load_element_stress_finetest_mesh.csv')));

    save(fullfile(outDir, 'q3_bonus_new_load_workspace_results.mat'), ...

        'par', 'meshList', 'solverType', 'essentialBC', ...

        'summary_table', 'nodal_table', 'element_table', ...

        'meshes', 'solutions', 'forces', 'errorData', 'stressData');

end

figure('Color', 'w');

loglog(h_mm_col, relative_L2_displacement_error_col, 'o-', 'Linewidth', 1.5,
'MarkerSize', 6); hold on;

loglog(h_mm_col, reference_relative_rms_error_col, 's-', 'Linewidth', 1.5,
'MarkerSize', 6);

set(gca, 'xDir', 'reverse');

grid on;

xlabel('h [mm]');

ylabel('Relative displacement error [-]');

title('Q3 bonus new load: spatial convergence');

legend('Relative L2 error vs exact field', 'Relative RMS error vs finest mesh',
'Location', 'best');

saveCurrentFigure(outDir, 'q3_bonus_new_load_spatial_convergence', saveFigures);

figure('Color', 'w');

plot(h_mm_col, free_end_u2_mm_col, 'o-', 'Linewidth', 1.5, 'MarkerSize', 6); hold on;

```

```

plot(h_mm_col, exact_free_end_u2_mm_col, 'k--', 'Linewidth', 1.5);

set(gca, 'xDir', 'reverse');

grid on;

xlabel('h [mm]');

ylabel('u_2(0,0) [mm]');

title('Q3 bonus new load: free-end displacement');

legend('Finite element', 'Exact field value', 'Location', 'best');

saveCurrentFigure(outDir, 'q3_bonus_new_load_free_end_displacement', saveFigures);


plotDeformedMesh(mesh, U, par, deformationScale, outDir, saveFigures);

plotNodalContour(mesh, U(1:2:end), 'u_1 [mm]', 'Q3 bonus new load: displacement u_1',
'q3_bonus_new_load_contour_u1', outDir, saveFigures);

plotNodalContour(mesh, U(2:2:end), 'u_2 [mm]', 'Q3 bonus new load: displacement u_2',
'q3_bonus_new_load_contour_u2', outDir, saveFigures);

plotNodalContour(mesh, F(2:2:end), 'F_2 [N]', 'Q3 bonus new load: nodal force F_2',
'q3_bonus_new_load_nodal_force_F2', outDir, saveFigures);

plotElementContour(mesh, elemStress.sigma1, '\sigma_1 [N/mm^2]', 'Q3 bonus new load:
element stress \sigma_1', 'q3_bonus_new_load_contour_sigma1', outDir, saveFigures);

plotElementContour(mesh, elemStress.tau12, '\tau_{12} [N/mm^2]', 'Q3 bonus new load:
element stress \tau_{12}', 'q3_bonus_new_load_contour_tau12', outDir, saveFigures);


function mesh = createRectMesh(L, a, Nx, Ny)

    x = linspace(0, L, Nx + 1);

    y = linspace(-a, a, Ny + 1);

    [x1, x2] = ndgrid(x, y);

```

```

coords = [X1(:), x2(:)];

nodeId = @(i, j) i + 1 + j * (Nx + 1);

elem = zeros(Nx * Ny, 4);

e = 0;

for j = 0:Ny - 1

    for i = 0:Nx - 1

        e = e + 1;

        elem(e, :) = [nodeId(i, j), nodeId(i + 1, j), nodeId(i + 1, j + 1),
nodeId(i, j + 1)];

    end

end

mesh.coords = coords;

mesh.elem = elem;

mesh.Nx = Nx;

mesh.Ny = Ny;

mesh.nNode = size(coords, 1);

mesh.nElem = size(elem, 1);

mesh.nodeId = nodeId;

mesh.xgrid = x;

mesh.ygrid = y;

mesh.hx = L / Nx;

```

```

mesh.hy = 2 * a / Ny;

mesh.L = L;

mesh.a = a;

end

function [K, F] = assembleSystem(mesh, par)

    ndof = 2 * mesh.nNode;

    F = zeros(ndof, 1);

    Iidx = zeros(64 * mesh.nElem, 1);
    Jidx = zeros(64 * mesh.nElem, 1);
    vals = zeros(64 * mesh.nElem, 1);
    counter = 0;

    gp = [-1 / sqrt(3), -1 / sqrt(3);
          1 / sqrt(3), -1 / sqrt(3);
          1 / sqrt(3), 1 / sqrt(3);
          -1 / sqrt(3), 1 / sqrt(3)];

    gw = [1; 1; 1; 1];

    for e = 1:mesh.nElem

        nodes = mesh.elem(e, :);

        Xe = mesh.coords(nodes, :);

        Ke = zeros(8, 8);

```

```

for q = 1:4

    xi = gp(q, 1);

    eta = gp(q, 2);

    [~, dN_dxi] = shapeQ4(xi, eta);

    Jmat = Xe.' * dN_dxi;

    detJ = det(Jmat);

    if detJ <= 0

        error('Element %d has non-positive Jacobian determinant.', e);

    end

    dN_dx = dN_dxi / Jmat;

    B = buildB(dN_dx);

    Ke = Ke + par.b * (B.' * par.D * B) * detJ * gw(q);

end

edofs = elementDofs(nodes);

for ii = 1:8

    for jj = 1:8

        counter = counter + 1;

        Iidx(counter) = edofs(ii);

        Jidx(counter) = edofs(jj);

        vals(counter) = Ke(ii, jj);

```

```

        end

    end

end

K = sparse(Iidx, Jidx, vals, ndof, ndof);

K = 0.5 * (K + K. ');

end

function F = applyUniformFreeEndLoad(F, mesh, par)

    nodalEndForce = par.P / (2 * mesh.Ny);

    for j = 0:mesh.Ny - 1

        n1 = mesh.nodeId(0, j);

        n4 = mesh.nodeId(0, j + 1);

        F(2 * n1) = F(2 * n1) + nodalEndForce;

        F(2 * n4) = F(2 * n4) + nodalEndForce;

    end

end

function [fixedDofs, fixedValues] = makeEssentialBC(mesh, par, essentialBC)

    tol = 1.0e-10 * max([1.0, mesh.L, mesh.a]);

    fixedNodes = find(abs(mesh.coords(:, 1) - par.L) <= tol);

    fixedDofs = zeros(2 * numel(fixedNodes), 1);

    fixedValues = zeros(2 * numel(fixedNodes), 1);

```

```

for k = 1:numel(fixedNodes)

    node = fixedNodes(k);

    fixedDofs(2 * k - 1) = 2 * node - 1;

    fixedDofs(2 * k) = 2 * node;

end

switch lower(essentialBC)

    case 'zero_clamp'

        fixedValues(:) = 0;

    case 'exact_displacement'

        [u1, u2] = exactDisplacement(mesh.coords(fixedNodes, 1),
mesh.coords(fixedNodes, 2), par);

        for k = 1:numel(fixedNodes)

            fixedValues(2 * k - 1) = u1(k);

            fixedValues(2 * k) = u2(k);

        end

    otherwise

        error('Unknown essentialBC. ');

    end

end
end

```

```

function [U, solverInfo] = solveWithEssentialBC(K, F, fixedDofs, fixedValues,
solverType)

    allDofs = (1:numel(F)).';

    fixedDofs = fixedDofs(:);

    fixedValues = fixedValues(:);

    freeDofs = setdiff(allDofs, fixedDofs);

    U = zeros(numel(F), 1);

    U(fixedDofs) = fixedValues;

    rhs = F(freeDofs) - K(freeDofs, fixedDofs) * fixedValues;

    Kff = K(freeDofs, freeDofs);

    [Ufree, solverInfo] = solveLinearSystem(Kff, rhs, solverType);

    U(freeDofs) = Ufree;

end

function [x, solverInfo] = solveLinearSystem(A, b, solverType)

    solverInfo.method = solverType;

    solverInfo.flag = NaN;

    solverInfo.relres = NaN;

    solverInfo.iter = NaN;

    solverInfo.chol_flag = NaN;

    switch lower(solverType)

```



```

case 'chol'

    p = symamd(A);

    Ap = A(p, p);

    bp = b(p);

    [R, cholFlag] = chol(Ap, 'upper');

    solverInfo.chol_flag = cholFlag;

    if cholFlag == 0

        xp = R \ (R.' \ bp);

        x = zeros(size(b));

        x(p) = xp;

        solverInfo.flag = 0;

        solverInfo.relres = norm(A * x - b) / max(norm(b), eps);

        solverInfo.iter = 1;

    else

        [x, solverInfoPCG] = solveLinearSystem(A, b, 'pcg');

        solverInfo.flag = solverInfoPCG.flag;

        solverInfo.relres = solverInfoPCG.relres;

        solverInfo.iter = solverInfoPCG.iter;

    end

case 'pcg'

    tol = 1.0e-10;

    maxit = max(200, min(size(A, 1), 5000));

```

```

try

    opts.type = 'ict';

    opts.droptol = 1.0e-3;

    opts.diagcomp = 1.0e-8;

    L = ichol(A, opts);

    [x, flag, relres, iter] = pcg(A, b, tol, maxit, L, L.');
```

```

catch

    [x, flag, relres, iter] = pcg(A, b, tol, maxit);
```

```

end
```

```

solverInfo.flag = flag;

solverInfo.relres = relres;

solverInfo.iter = iter;

solverInfo.chol_flag = NaN;
```

```

case 'backslash'
```

```

    x = A \ b;

    solverInfo.flag = 0;

    solverInfo.relres = norm(A * x - b) / max(norm(b), eps);

    solverInfo.iter = 1;

    solverInfo.chol_flag = NaN;
```

```

otherwise
```

```

    error('Unknown solverType.');
```

end

end

```
function err = computeErrorNorms(mesh, U, par)
```

```
    coords = mesh.coords;
```

```
    Uxy = [U(1:2:end), U(2:2:end)];
```

```
    [u1Exact, u2Exact] = exactDisplacement(coords(:, 1), coords(:, 2), par);
```

```
    nodalErr = Uxy - [u1Exact, u2Exact];
```

```
    nodalErrNorm = sqrt(sum(nodalErr.^2, 2));
```

```
    fixedMask = abs(coords(:, 1) - par.L) <= 1.0e-10 * max([1.0, par.L, par.a]);
```

```
    interiorMask = ~fixedMask;
```

```
    gp = [-1 / sqrt(3), -1 / sqrt(3);
```

```
          1 / sqrt(3), -1 / sqrt(3);
```

```
          1 / sqrt(3), 1 / sqrt(3);
```

```
          -1 / sqrt(3), 1 / sqrt(3)];
```

```
    gw = [1; 1; 1; 1];
```

```
    intErr2 = 0;
```

```
    intExact2 = 0;
```

```
    intStressErr2 = 0;
```

```
    intStressExact2 = 0;
```

```

for e = 1:mesh.nElem

    nodes = mesh.elem(e, :);

    Xe = coords(nodes, :);

    edofs = elementDofs(nodes);

    Ue = U(edofs);

    for q = 1:4

        xi = gp(q, 1);

        eta = gp(q, 2);

        [N, dN_dxi] = shapeQ4(xi, eta);

        Jmat = Xe.' * dN_dxi;

        detJ = det(Jmat);

        dN_dx = dN_dxi / Jmat;

        Nmat = buildNmat(N);

        B = buildB(dN_dx);

        xq = N.' * Xe(:, 1);

        yq = N.' * Xe(:, 2);

        uh = Nmat * Ue;

        strainh = B * Ue;

        stressh = par.D * strainh;

```

```

        [u1e, u2e] = exactDisplacement(xq, yq, par);

        [s1e, s2e, t12e] = exactStress(xq, yq, par);

        ue = [u1e; u2e];

        stresse = [s1e; s2e; t12e];

        intErr2 = intErr2 + sum((uh - ue).^2) * detJ * gw(q);

        intExact2 = intExact2 + sum(ue.^2) * detJ * gw(q);

        intStressErr2 = intStressErr2 + sum((stressh - stresse).^2) * detJ *
gw(q);

        intStressExact2 = intStressExact2 + sum(stresse.^2) * detJ * gw(q);

    end

end

err.maxNodalDispError = max(nodalErrNorm);

err.rmsNodalDispError = sqrt(mean(nodalErrNorm.^2));

err.maxInteriorNodalDispError = max(nodalErrNorm(interiorMask));

err.L2DispError = sqrt(intErr2);

err.relL2DispError = sqrt(intErr2 / max(intExact2, eps));

err.relL2StressError = sqrt(intStressErr2 / max(intStressExact2, eps));

end

function elemStress = computeElementStress(mesh, U, par)

    elemStress.x1Center = zeros(mesh.nElem, 1);

    elemStress.x2Center = zeros(mesh.nElem, 1);

```

```

elemStress.sigma1 = zeros(mesh.nElem, 1);

elemStress.sigma2 = zeros(mesh.nElem, 1);

elemStress.tau12 = zeros(mesh.nElem, 1);

elemStress.vonMises = zeros(mesh.nElem, 1);

elemStress.sigma1Exact = zeros(mesh.nElem, 1);

elemStress.sigma2Exact = zeros(mesh.nElem, 1);

elemStress.tau12Exact = zeros(mesh.nElem, 1);

elemStress.vonMisesExact = zeros(mesh.nElem, 1);


for e = 1:mesh.nElem

    nodes = mesh.elem(e, :);

    xe = mesh.coords(nodes, :);

    ue = U(elementDofs(nodes));


    [N, dN_dxi] = shapeQ4(0.0, 0.0);

    Jmat = xe.' * dN_dxi;

    dN_dx = dN_dxi / Jmat;

    B = buildB(dN_dx);


    xq = N.' * xe(:, 1);

    yq = N.' * xe(:, 2);


    stress = par.D * B * ue;

    [s1e, s2e, t12e] = exactStress(xq, yq, par);

```

```

    elemStress.x1Center(e) = xq;

    elemStress.x2Center(e) = yq;

    elemStress.sigma1(e) = stress(1);

    elemStress.sigma2(e) = stress(2);

    elemStress.tau12(e) = stress(3);

    elemStress.vonMises(e) = planeStressVonMises(stress(1), stress(2), stress(3));

    elemStress.sigma1Exact(e) = s1e;

    elemStress.sigma2Exact(e) = s2e;

    elemStress.tau12Exact(e) = t12e;

    elemStress.vonMisesExact(e) = planeStressVonMises(s1e, s2e, t12e);

end

end

function [N, dN_dxi] = shapeQ4(xi, eta)

    N = 0.25 * [(1 - xi) * (1 - eta);

                (1 + xi) * (1 - eta);

                (1 + xi) * (1 + eta);

                (1 - xi) * (1 + eta)];

    dN_dxi = 0.25 * [-(1 - eta), -(1 - xi);

                    (1 - eta), -(1 + xi);

                    (1 + eta), (1 + xi);

                    -(1 + eta), (1 - xi)];

end

```

```

function B = buildB(dN_dx)

    B = zeros(3, 8);

    for i = 1:4

        c1 = 2 * i - 1;

        c2 = 2 * i;

        B(1, c1) = dN_dx(i, 1);

        B(2, c2) = dN_dx(i, 2);

        B(3, c1) = dN_dx(i, 2);

        B(3, c2) = dN_dx(i, 1);

    end

end

function Nmat = buildNmat(N)

    Nmat = zeros(2, 8);

    for i = 1:4

        Nmat(1, 2 * i - 1) = N(i);

        Nmat(2, 2 * i) = N(i);

    end

end

function dofs = elementDofs(nodes)

    nodes = nodes(:);

```



```

dofs = zeros(8, 1);

dofs(1:2:end) = 2 * nodes - 1;

dofs(2:2:end) = 2 * nodes;

end

function [u1, u2] = exactDisplacement(x1, x2, par)

    P = par.P;

    L = par.L;

    a = par.a;

    E = par.E;

    I = par.I;

    nu = par.nu;

    G = par.G;

    u1 = -P ./ (2 * E * I) .* (x1.^2 - L^2) .* x2 ...

        + nu * P ./ (6 * E * I) .* x2.^3 ...

        - P ./ (2 * G * I) .* (a^2 .* x2 - x2.^3 ./ 3);

    u2 = P ./ (6 * E * I) .* (x1.^3 - 3 * L^2 .* x1 + 2 * L^3) ...

        + nu * P ./ (2 * E * I) .* (L - x1) .* x2.^2 ...

        + P * a^2 ./ (2 * G * I) .* (L - x1);

end

function [sigma1, sigma2, tau12] = exactStress(x1, x2, par)

    P = par.P;

```

```

a = par.a;

I = par.I;

sigma1 = -P .* x1 .* x2 ./ I;

sigma2 = zeros(size(x1));

tau12 = -P ./ (2 * I) .* (a^2 - x2.^2);

end

function vm = planeStressvonMises(sigma1, sigma2, tau12)

    vm = sqrt(sigma1.^2 - sigma1 .* sigma2 + sigma2.^2 + 3 * tau12.^2);

end

function val = interpolatePointDisplacement(mesh, U, xq, yq, component)

    Uq = interpolateSolutionAtPoints(mesh, U, [xq, yq]);

    val = Uq(1, component);

end

function Uq = interpolateSolutionAtPoints(mesh, U, queryCoords)

    nq = size(queryCoords, 1);

    Uq = zeros(nq, 2);

    for q = 1:nq

        xq = queryCoords(q, 1);

        yq = queryCoords(q, 2);

```

```

i = floor((xq - mesh.xgrid(1)) / mesh.hx);

j = floor((yq - mesh.ygrid(1)) / mesh.hy);


i = max(0, min(mesh.Nx - 1, i));

j = max(0, min(mesh.Ny - 1, j));


xLeft = mesh.xgrid(i + 1);

yBottom = mesh.ygrid(j + 1);


xi = 2 * (xq - xLeft) / mesh.hx - 1;

eta = 2 * (yq - yBottom) / mesh.hy - 1;


xi = max(-1, min(1, xi));

eta = max(-1, min(1, eta));


[N, ~] = shapeQ4(xi, eta);


nodes = [mesh.nodeId(i, j), mesh.nodeId(i + 1, j), ...

          mesh.nodeId(i + 1, j + 1), mesh.nodeId(i, j + 1)];


ux = U(2 * nodes - 1);

uy = U(2 * nodes);


Uq(q, 1) = N.' * ux(:);

Uq(q, 2) = N.' * uy(:);

```

```
end
```

```
end
```

```
function rate = convergenceRate(h, err)
```

```
    n = numel(h);
```

```
    rate = NaN(n, 1);
```

```
    for i = 2:n
```

```
        if err(i - 1) > 0 && err(i) > 0 && h(i - 1) > h(i)
```

```
            rate(i) = log(err(i - 1) / err(i)) / log(h(i - 1) / h(i));
```

```
        end
```

```
    end
```

```
end
```

```
function c = safeCondest(A)
```

```
    try
```

```
        c = condest(A);
```

```
    catch
```

```
        c = NaN;
```

```
    end
```

```
end
```

```
function plotDeformedMesh(mesh, U, par, scaleFactor, outDir, saveFigures)
```

```
    Uxy = [U(1:2:end), U(2:2:end)];
```

```
    maxDisp = max(sqrt(sum(Uxy.^2, 2)));
```

```

if isempty(scaleFactor)

    scaleFactor = 0.2 * par.L / max(maxDisp, eps);

end

defCoords = mesh.coords + scaleFactor * Uxy;

figure('color', 'w');

patch('Faces', mesh.elem, 'Vertices', mesh.coords, ...

    'FaceColor', 'none', 'EdgeColor', [0.65 0.65 0.65], 'Linewidth', 0.6);

hold on;

patch('Faces', mesh.elem, 'Vertices', defCoords, ...

    'FaceColor', 'none', 'EdgeColor', [0.00 0.20 0.80], 'Linewidth', 0.8);

grid on;

axis equal tight;

xlabel('x_1 [mm]');

ylabel('x_2 [mm]');

title(sprintf('Q3 bonus new load: undeformed and deformed mesh, scale = %.4g',
scaleFactor));

legend('Undeformed mesh', 'Deformed mesh', 'Location', 'best');

saveCurrentFigure(outDir, 'q3_bonus_new_load_deformed_mesh', saveFigures);

end

function plotNodalContour(mesh, nodalValue, colorbarLabel, figTitle, fileName, outDir,
saveFigures)

```

```

figure('Color', 'w');

patch('Faces', mesh.elem, 'Vertices', mesh.coords, ...

    'FaceVertexCData', nodalValue(:), 'FaceColor', 'interp', ...

    'EdgeColor', [0.70 0.70 0.70], 'Linewidth', 0.3);

axis equal tight;

grid on;

xlabel('x_1 [mm]');

ylabel('x_2 [mm]');

title(figTitle);

cb = colorbar;

ylabel(cb, colorbarLabel);

saveCurrentFigure(outDir, fileName, saveFigures);

end

function plotElementContour(mesh, elemValue, colorbarLabel, figTitle, fileName,
outDir, saveFigures)

figure('Color', 'w');

patch('Faces', mesh.elem, 'Vertices', mesh.coords, ...

    'FaceColor', 'flat', 'CData', elemValue(:), ...

    'EdgeColor', 'none');

axis equal tight;

grid on;

xlabel('x_1 [mm]');

ylabel('x_2 [mm]');

title(figTitle);

```

```
    cb = colorbar;

    ylabel(cb, colorbarLabel);

    saveCurrentFigure(outDir, fileName, saveFigures);

end

function saveCurrentFigure(outDir, fileName, saveFigures)

    if saveFigures

        try

            exportgraphics(gcf, fullfile(outDir, [fileName, '.png']), 'Resolution',
300);

        catch

            saveas(gcf, fullfile(outDir, [fileName, '.png']));

        end

        savefig(gcf, fullfile(outDir, [fileName, '.fig']));

    end

end
```